



Jomo Kenyatta University of Agriculture and Technology

College of Engineering and Technology

School of Mechanical, Materials, and Manufacturing Engineering

Department of Mechatronic Engineering

Modification and Improvement of a Coffee Berry Sorting Machine

(FYP 22-22)

Interim Report

Joe Annel (ENM221-0051/2021)

Prosper Muuo (ENM221-0218/2021)

Supervisor

Dr. Maurine Andanje

April 7, 2026

Declaration

We, the undersigned, declare that this Interim Report is our original work undertaken in partial fulfillment of the requirements for the award of the Bachelor of Science degree in Mechatronic Engineering from Jomo Kenyatta University of Agriculture and Technology. This report has not been submitted for any degree or examination in any other institution.

Joe Annel

ENM221-0051/2021

Signature: _____

Date: _____

Prosper Muuo

ENM221-0218/2021

Signature: _____

Date: _____

This Interim Report has been submitted with my approval as University Supervisor:

Dr. Maureen Andanje

Department of Mechatronic Engineering

Jomo Kenyatta University of Agriculture and Technology

Signature: _____

Date: _____

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Dr. Maurine Andanje, for her invaluable guidance, encouragement, and technical insights throughout this project. Her constructive feedback has been instrumental in shaping both the direction and quality of this work.

We also extend our appreciation to the Department of Mechatronic Engineering at Jomo Kenyatta University of Agriculture and Technology for providing the resources and facilities necessary for conducting this research.

Special thanks to the previous group whose foundational work on the coffee berry sorting machine provided the starting point for our modifications and improvements.

Finally, we acknowledge the support of our families and colleagues who provided encouragement throughout the course of this project.

Abstract

Automated coffee berry sorting is a critical step in improving the quality and efficiency of coffee processing, particularly for small- and medium-scale producers in Kenya. This project addresses the limitations of an existing image-based sorting machine developed by fifth-year Mechatronics Engineering students at JKUAT, which employed a Raspberry Pi vision system and air-ejection actuation. Key challenges identified include processing latency, motion blur under dynamic conditions, and insufficient classification throughput.

This interim report presents the work completed to date, covering a review of relevant literature on computer vision-based sorting systems, the proposed methodology for improving the system, and progress achieved. A custom YOLOv11 object detection model has been trained on a dataset of harvested coffee berry images, targeting four ripeness classes: unripe, semi-ripe, ripe, and overripe. Initial evaluation results demonstrate promising detection performance, with the model achieving competitive mean average precision (mAP) values at moderate inference speeds.

The remaining work focuses on integrating the trained model with the physical sorting machine, validating real-time performance on the vibrating system, and optimizing the detection-to-actuation pipeline for improved throughput and reduced latency. A detailed work plan for the completion of remaining objectives is also presented.

Contents

Declaration	I
Acknowledgments	II
Abstract	III
1 Introduction	1
1.1 Background of Study	1
1.2 Problem Statement	2
1.3 Objectives	4
1.3.1 Main Objective	4
1.3.2 Specific Objectives	4
1.4 Justification	4
1.5 Scope of the Project	5
1.6 Expected Outcomes	5
2 Literature Review	7
2.1 Challenges of Manual and Early Automated Sorting	8
2.2 Mechanical Material Handling	8
2.3 Camera Technology: Rolling Shutter vs. Global Shutter	10
2.4 Deep Learning Object Detection for Sorting	11
2.4.1 CNN-Based Feature Extraction	11
2.4.2 MobileNetV2 and its Limitations	11
2.4.3 YOLO Family: Real-Time Detection with Localisation	12
2.4.4 YOLOv11 on Embedded Platforms	13
2.5 Binary (Accept/Reject) vs. Fine-Grained Classification	13
2.6 Embedded Co-Processor Architectures	15
2.7 Previous Works	16
2.7.1 System Architecture of the Reference Machine	16

2.7.2	Identified Limitations of the Reference Machine	17
2.8	Summary of Literature Gaps Addressed by This Project	20
3	Methodology	22
3.1	Performance Analysis of Existing Machine	22
3.2	Mechanical Subsystem — Retained and Fabricated	23
3.2.1	Berry Feeding Mechanism	24
3.2.2	Inspection Chute and Ejection	25
3.2.3	Structural Frame and Components	27
3.3	Dataset Collection and Model Training	30
3.3.1	Image Acquisition	30
3.3.2	Annotation	31
3.3.3	Dataset Split	32
3.4	YOLOv11 Model Training	32
3.4.1	Framework and Backbone	32
3.4.2	Training Configuration	33
3.4.3	Evaluation Metrics	33
3.5	Sorting Mechanism Improvements	33
3.5.1	Hardware Platform	34
3.5.2	Detection-to-Actuation Pipeline	37
3.5.3	ESP32 Pin Assignment and Wiring	39
3.5.4	Transistor Technology Selection: BJT vs. Logic-Level MOSFET . .	41
3.5.5	Solenoid Driver Circuit	42
3.6	Circuit Simulation in Proteus	43
3.6.1	Simulation Scope	44
3.6.2	Simulation Methodology	44
3.7	Validation Strategy	45
4	Results and Discussion	46

4.1	Overview	46
4.2	Design Rationale	46
4.2.1	Design Drawings and Assembly Outputs	46
4.3	Sorting Mechanism Improvements	47
4.3.1	Procured and Verified Hardware	48
4.3.2	Proteus Circuit Simulation Results	49
4.4	Model Training Results	52
4.4.1	Dataset and Class Reduction	52
4.4.2	YOLOv11n Training Results	53
4.4.3	Dataset Visualisations	56
4.4.4	Sample Detection Output	58
4.5	Software Development	59
4.6	Discussion: Progress Against Objectives	62
5	Conclusion	63
5.1	Summary	63
5.2	Future Work	64
5.3	Project Schedule	64
	Appendices	69
	Appendix A: Circuit Schematics	69
	Appendix B: Schedule and Budget	73
	Appendix C: Dataset and Model Outputs	75
	Appendix D: Mechanical Design	76

List of Figures

1.1	Harvested coffee berry variation showing visual diversity in colour, size, and surface texture	2
2.1	Ripe coffee berries showing colour variation from green to deep red	7
2.2	Comparison of manual and electronic coffee sorting methods	8
2.3	Fully fabricated mechanical assembly from the reference implementation .	9
2.4	CAD drawing of the air-ejector nozzle assembly	10
2.5	Image processing pipeline for YOLOv11-based sorter on Raspberry Pi 4: frame capture (global shutter) → pre-processing → YOLOv11 inference → classification (accept/reject) → ejection control	12
2.6	Overview of classification approaches considered: classical machine learning, CNN-based models, and single-stage detectors (YOLO family)	13
2.7	Visual comparison of ripe (accept) and unripe/semi-ripe (reject) coffee berries. The binary classification boundary maps to the deep-red colour transition.	14
2.8	Proposed co-processing architecture	16
2.9	Reference model validation predictions showing four-class classification (unripe, semi-ripe, ripe, overripe) with visible confusion between visually similar categories	19
2.10	Reference model F1-score performance across the four ripeness categories, highlighting the classification difficulty at the semi-ripe/ripe boundary . . .	20
3.1	Overall architecture of the improved coffee berry sorting machine	22
3.2	Hopper assembly — berry loading and gravity feed into the trough	24
3.3	Vibrating feeder trough — singulated berry stream to inspection zone . . .	25
3.4	Inspection chute — berry transit path from feeder to ejection zone	26
3.5	Full assembly of the sorting machine	27
3.6	Main structural frame of the sorting machine	28
3.7	Full assembly front view	29

3.8	Co-processing architecture	35
3.9	System control flow: task distribution between Raspberry Pi 4 (inference and tracking) and ESP32 (actuation, timing, and PWM control)	36
3.10	Image processing pipeline for the YOLOv11-based sorter: frame capture (global shutter) → pre-processing → YOLOv11 inference on Raspberry Pi 4 → centroid tracking → UART command to ESP32	38
3.11	UART communication and data flow between Raspberry Pi 4 and ESP32: the Pi sends <code>REJECT:n</code> or <code>ACCEPT</code> commands after each inference decision; the ESP32 handles all real-time solenoid actuation.	39
3.12	Fritzing circuit diagram of the ESP32 solenoid driver implementation: IRLZ44N MOSFET switching stages, flyback diodes, gate resistors, and Raspberry Pi UART connection	43
4.1	Mechanical design drawings used for fabrication and assembly alignment .	47
4.2	System block diagram: processor hierarchy, communication links, and actuator outputs with power distribution	49
4.3	Complete Proteus ISIS simulation: ESP32 with four IRLZ44N solenoid driver channels, vibrating motor PWM, LED strip driver, 2-channel optocoupler relay, and UART virtual terminal	50
4.4	Proteus oscilloscope: MOSFET turn-ON — gate rises to 3.3 V and drain falls from 12 V to ≈ 0 V with no ringing (IRLZ44N, 220 Ω gate resistor, 100 k Ω pull-down)	51
4.5	Proteus oscilloscope: MOSFET turn-OFF — drain recovers to 12 V; 1N5819 clamps inductive kickback spike within 0.5 V of the supply rail	51
4.6	YOLOv11n training metrics over 15 epochs showing loss, precision, recall, and mAP on the binary coffee berry dataset	53
4.7	F1-score vs. confidence threshold curve for the YOLOv11n binary classifier, showing optimal operating point for accept/reject classification	54
4.8	Precision-Recall curve demonstrating the trade-off between precision and recall across confidence thresholds for both accept and reject classes	55

4.9	Normalised confusion matrix on the test split showing classification accuracy for accept (ripe) and reject (unripe/semi-ripe/overripe) categories . . .	56
4.10	Ground truth label distribution visualisation showing the spatial distribution and class balance of accept and reject instances across the training dataset	57
4.11	Validation batch 0 with ground-truth bounding box annotations for accept (ripe) and reject (unripe/semi-ripe) berries	58
4.12	YOLOv11n inference output on validation batch 0: predicted accept/reject bounding boxes with confidence scores demonstrating the model's classification performance on unseen test images	59
4.13	System control flow: Pi 4 handles image processing and YOLOv11 inference; ESP32 handles motor control, timing, and actuation	61
.1	Electrical schematic	69
.2	ESP32 pin mapping schematic	70
.3	Circuit implementation layout (Fritzing)	71
.4	Proteus full-circuit simulation	72
.5	Project Gantt chart and schedule	73
.6	Reference model F1 curve	75
.7	Reference model validation batch labels	76
.8	Base plate technical drawing with dimensions and specifications	77
.9	Chute assembly technical drawing with dimensions and specifications . . .	78
.10	Chute mounting bracket technical drawing with dimensions and specifications	79
.11	Vibrating feeder trough technical drawing with dimensions and specifications	80
.12	Hopper assembly technical drawing with dimensions and specifications . .	80

List of Tables

2.1	Literature gaps and corresponding improvements in this project	21
3.1	Image dataset sources used for model training	31
3.2	Binary coffee berry classification scheme	32
3.3	YOLOv11 training hyperparameters	33
3.4	Raspberry Pi, ESP32, and Peripheral Wiring Assignment	40
3.5	BJT (PN2222A) vs. Logic-Level MOSFET (IRLZ44N) switching comparison	41
4.1	Dataset split summary (2,000-image binary dataset)	53
4.2	YOLOv11n evaluation results on test split	56
4.3	Progress summary against specific objectives	62
.1	Project Budget — Modification and Improvement of a Coffee Berry Sorting Machine	74

1 Introduction

1.1 Background of Study

The global coffee industry remains a significant contributor to the economies of many developing countries, supporting millions of smallholder farmers worldwide. Coffee production involves several critical stages, from cultivation and harvesting to processing and packaging. Among these stages, the sorting of coffee berries is one of the most labor-intensive and quality-sensitive processes [1]. Traditionally, coffee berry sorting has relied heavily on manual visual inspection, where workers separate berries based on ripeness, size, and visible defects. This approach is time-consuming, inconsistent, and costly, often leading to variations in coffee quality and increased operational expenses [2].

Manual sorting methods are highly susceptible to human error and fatigue, which can result in misclassification of berries and reduced overall product quality [3]. Furthermore, the labor-intensive nature of manual sorting limits scalability and efficiency, particularly during peak harvest periods when labor shortages are common [4]. These challenges highlight the need for automated sorting solutions capable of performing classification tasks with higher accuracy, consistency, and throughput [1].

Recent advancements in computer vision, machine learning, and embedded systems have enabled the development of automated coffee berry sorting machines that utilize image-based detection and real-time classification techniques. Previous work by fifth-year mechatronics engineering students resulted in the development of an image-based coffee berry sorting machine incorporating a vibrating system, optical detection, and air-ejection mechanisms controlled by a Raspberry Pi-based processing unit. While the system successfully demonstrated automated sorting, practical challenges were identified during real-time operation, particularly related to motion blur, processing latency, and limited computational throughput.



Figure 1.1: Harvested coffee berry variation showing visual diversity in colour, size, and surface texture [5]

The visual diversity in harvested coffee berries—including variations in colour, size, and surface texture, as illustrated in Figure 1.1—presents a significant challenge for accurate automated classification.

This project proposes the modification and improvement of the existing coffee berry sorting machine by addressing the identified limitations in image acquisition, processing speed, and real-time classification performance. By enhancing the vision processing pipeline, optimizing algorithms, and evaluating alternative hardware or processing architectures, the project aims to improve sorting accuracy and throughput while maintaining a cost-effective and scalable design.

1.2 Problem Statement

The quality of coffee berries at the processing stage directly determines the final product grade, market value, and competitiveness in both local and international markets. Despite

the successful demonstration of automated sorting in the previous system, it exhibits several critical technical limitations:

1. **Motion blur:** The existing Raspberry Pi Camera V2 uses a rolling shutter, which exposes each row of pixels at a slightly different point in time. As berries travel along the chute at speeds of $0.4\text{--}0.6\text{ m s}^{-1}$, this row-by-row exposure introduces a smearing artefact that distorts the apparent colour, shape, and contour of each berry. Since colour is the primary classification feature, this degradation directly reduces model accuracy [6].
2. **Illumination inconsistency:** Variable ambient lighting causes identical berries to produce different colour values under different conditions. Without controlled, consistent illumination above the detection zone, the classification model encounters inputs that differ significantly from its training distribution, leading to erroneous accept/reject decisions [7].
3. **Processing latency:** The limited computational capacity of the Raspberry Pi 4 under a single-threaded pipeline causes delays across the frame acquisition, preprocessing, inference, and actuation pipeline, constraining throughput.
4. **Model accuracy and class confusion:** The original four-class model confused visually similar semi-ripe and ripe berries, causing commercially acceptable red berries to be rejected and semi-ripe berries to pass into the accept stream, reducing economic yield.

There is a clear need to improve the vision system, processing architecture, and real-time control strategy to achieve higher throughput, lower latency, and improved classification accuracy.

1.3 Objectives

1.3.1 Main Objective

To design, implement, and validate an improved coffee berry sorting machine that achieves $>90\%$ mAP@0.5, end-to-end latency below 100 ms, and throughput above 150 berries per minute.

1.3.2 Specific Objectives

1. To quantify baseline performance of the existing system (latency, accuracy, and throughput) and document root causes for each limitation.
2. To implement a global-shutter camera, MOSFET-based ejector drivers, and an ESP32 co-processor, and verify motion-blur reduction and solenoid timing repeatability (target jitter ≤ 5 ms).
3. To implement an accept/reject classification scheme using a Computer vision model, and evaluate performance on a held-out test set.
4. To test the improved system under real-world conditions and validate the performance gains.

1.4 Justification

Coffee remains one of Kenya's most valuable agricultural exports, and quality grading plays a major role in determining its economic value. Automated sorting systems offer a practical solution for improving quality consistency and reducing labour dependency. Improving the existing image-based system provides a cost-effective pathway by building upon already validated concepts while addressing observed technical shortcomings.

The proposed improvements are expected to:

1. Enhance accuracy and throughput in coffee berry sorting.
2. Enable better utilisation of affordable embedded vision technologies.
3. Increase the feasibility of locally developed agri-tech solutions.
4. Contribute practical learning and innovation in mechatronics system integration.

1.5 Scope of the Project

This project is limited to the modification and improvement of the existing coffee berry sorting machine. Specifically:

- The mechanical subsystem (vibratory feeder, hopper, chute, and air-ejection hardware) is retained and re-fabricated; only minor fitment changes are made to support improved camera alignment and ejector positioning.
- Focus is placed on the vision and control sub-systems: camera selection, lighting, model training, inference pipeline, and microcontroller–Pi communication.
- Classification uses a binary **ACCEPT/REJECT** scheme: the **accept** class covers uniform deep-red ripe berries; the **reject** class covers unripe (green), semi-ripe (yellow/orange), overripe (dark/black), and physically damaged berries. This aligns with Kenyan cooperative grading practice.
- Deployment targets the Raspberry Pi 4 or equivalent single-board computer for embedded inference.

1.6 Expected Outcomes

Upon completion of this project, the following outcomes are expected:

1. A documented quantitative baseline of the existing system's performance limitations, including measured end-to-end latency, overall classification accuracy, and throughput, with identified root causes for each limitation.
2. An upgraded sorting mechanism with a global-shutter camera eliminating motion blur artefacts, MOSFET ejector drivers achieving reliable switching at ESP32 3.3 V logic levels, and an ESP32 co-processor handling all hard real-time actuation tasks.
3. A YOLOv11n binary classifier deployed on Raspberry Pi 4 achieving $\text{mAP}@0.5 > 90\%$, end-to-end pipeline latency below 100 ms, and sorting throughput above 150 berries per minute.

2 Literature Review

The global coffee industry plays a critical role in the economies of many developing countries, including Kenya, where it provides livelihoods to millions of smallholder farmers. Coffee processing involves several critical stages, among which the sorting of coffee berries determines final product quality and market value. The presence of unripe, overripe, or defective berries significantly degrades cup quality, leading to price penalties and rejection by premium buyers [1, 2].

Automated sorting systems based on machine vision offer a path to consistent, high-throughput grading [8, 9]. A previous fifth-year project at this institution developed an image-based coffee berry sorting machine that demonstrated the feasibility of automated sorting but revealed significant technical gaps in real-time performance [5]. This chapter documents that prior system in detail, surveys the relevant literature across mechanical handling, vision hardware, deep learning models, and embedded architectures, and identifies the specific gaps addressed by this work.



Figure 2.1: Ripe coffee berries showing colour variation from green to deep red [5]

2.1 Challenges of Manual and Early Automated Sorting

1. **Labour Intensity and Fatigue:** Prolonged visual inspection leads to fatigue, reducing sorting accuracy and consistency [1]. Fatigued workers are more likely to misclassify berries, allowing defective berries to pass or rejecting acceptable ones.
2. **Human Error and Subjectivity:** Significant variability in sorting outcomes has been observed across different shifts, even when standardised grading guidelines were provided [2]. This subjectivity results in inconsistent quality between batches.
3. **Limited Throughput:** Human sorting speed becomes a bottleneck during peak harvest seasons [3]. Automated systems address this but introduce the challenges of motion blur, latency, and fine-grained class confusion documented above.

2.2 Mechanical Material Handling

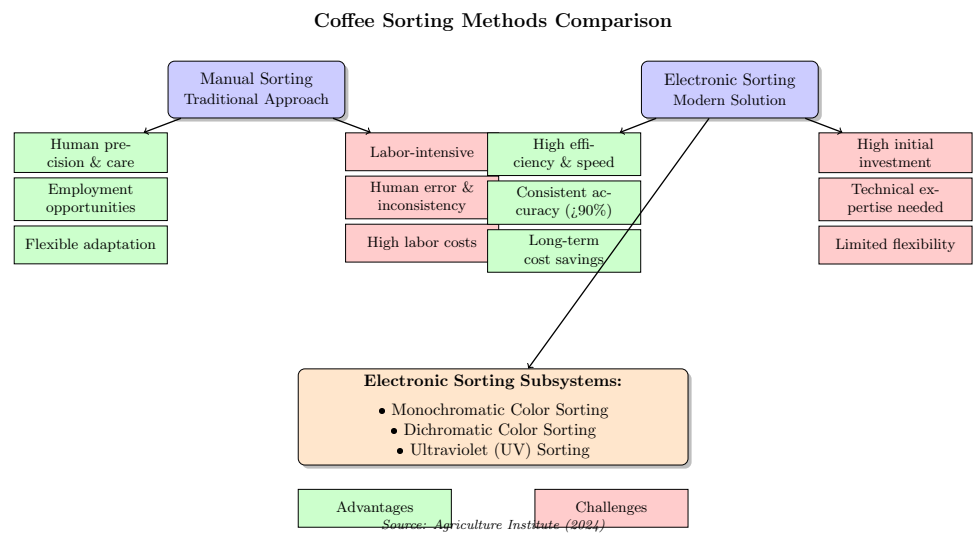


Figure 2.2: Comparison of manual and electronic coffee sorting methods

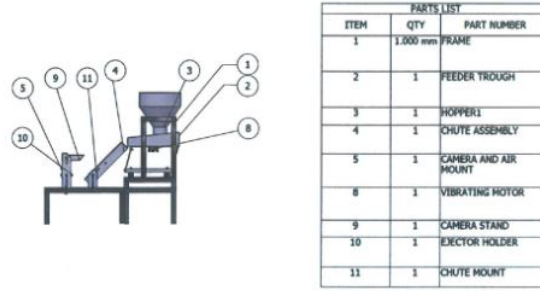


Figure 2.3: Fully fabricated mechanical assembly from the reference implementation [5], showing integrated hopper, vibrating feeder, chute, and frame structure. This mechanical subsystem forms the foundation upon which the improved imaging and control systems are built.

Vibrating feeder-chute systems are standard in agricultural sorting. The amplitude and frequency of an eccentric motor are tuned to prevent bridging while maintaining adequate throughput and berry singulation. Chute inclination angle and surface finish further influence flow rate and separation. Literature confirms that this approach produces reliably consistent berry spacing at throughputs of 100–300 berries per minute in laboratory prototypes [7].

The air-jet deflection mechanism, which the reference system uses for separation, operates on a detect-decide-deflect principle. Berries passing the imaging zone trigger compressed-air jets timed to deflect reject-class berries into separate chutes while accepted berries continue by gravity. Key advantages include minimal physical contact and high speed; limitations include compressed-air infrastructure requirements and tight timing constraints [10].

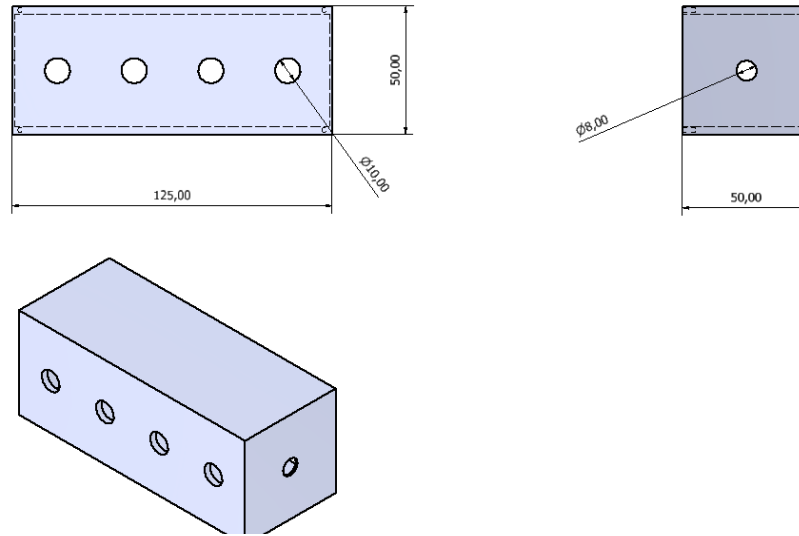


Figure 2.4: CAD drawing of the air-ejector nozzle assembly from the reference sorting machine showing solenoid valve, nozzle orientation, and berry deflection trajectory [5]

2.3 Camera Technology: Rolling Shutter vs. Global Shutter

Conventional CMOS cameras read out pixel rows sequentially (rolling shutter). For static scenes this is adequate, but when objects move faster than the sensor readout time, each row captures a slightly different object position, producing a characteristic shear distortion (jello effect) and edge smearing [4]. In the reference machine this was identified as the primary source of image degradation.

A **global-shutter** sensor exposes all pixels simultaneously, eliminating motion-induced distortion entirely. Global-shutter CMOS sensors (e.g., Sony IMX296, Omnivision OV9281) achieve effective shutter times of <1 ms at full resolution. At 0.5 m s^{-1} chute speed, a 1 ms exposure limits berry translation to 0.5 mm per frame—approximately $40\times$ less motion blur than the reference camera. This improvement is critical for reliable colour and texture classification at the ripe/reject boundary.

For this project, the **Arducam OV9281 global-shutter camera module** (Raspberry

Pi CSI-compatible, 1 MP, monochrome/colour variants) is proposed as the replacement imaging sensor, providing freeze-frame clarity even at the target operating chute speed.

2.4 Deep Learning Object Detection for Sorting

2.4.1 CNN-Based Feature Extraction

Deep convolutional neural networks (CNNs) automatically learn hierarchical features from labelled image data, eliminating the need for hand-crafted descriptors [11, 12]. For agricultural sorting, CNNs have consistently outperformed classical colour-histogram and SVM-based approaches [13], particularly under variable illumination [4]. Zhang et al. [6] demonstrated that improved deep convolutional architectures could identify disease symptoms on maize leaves with high accuracy under field conditions, highlighting the robustness of learned features across agricultural imaging tasks. A comprehensive review by Li et al. [8] and Kamilaris and Prenafeta-Boldú [9] further confirm that deep learning now dominates agricultural product quality grading, consistently outperforming traditional machine-learning pipelines on colour, texture, and shape discrimination tasks [14, 15].

2.4.2 MobileNetV2 and its Limitations

MobileNetV2 [16, 17] uses inverted residual bottleneck blocks with depthwise-separable convolutions to minimise parameter count for mobile deployment. When quantised to INT8 and exported as TFLite, MobileNetV2 achieves inference times of 80–120 ms on the Raspberry Pi 4. While fast enough in isolation, the reference system could not overlap inference with the next frame capture, making the pipeline strictly sequential and throughput-limited. Additionally, MobileNetV2 is a whole-image classifier: it does not produce bounding boxes, so when multiple berries appear in one frame it assigns a single class label to the entire image, causing mixed-berry frames to be misclassified. These limitations motivated the reference team’s migration to YOLOv11n, but the serialised pipeline on the Pi 3B+ remained the throughput bottleneck.

2.4.3 YOLO Family: Real-Time Detection with Localisation

YOLO (You Only Look Once) [18] reformulated object detection as a single regression problem, outputting bounding-box coordinates and class probabilities in a single forward pass. Successive versions (YOLOv3 [19], YOLOv5 [20], YOLOv8 [21]) progressively improved small-object recall and speed [22].

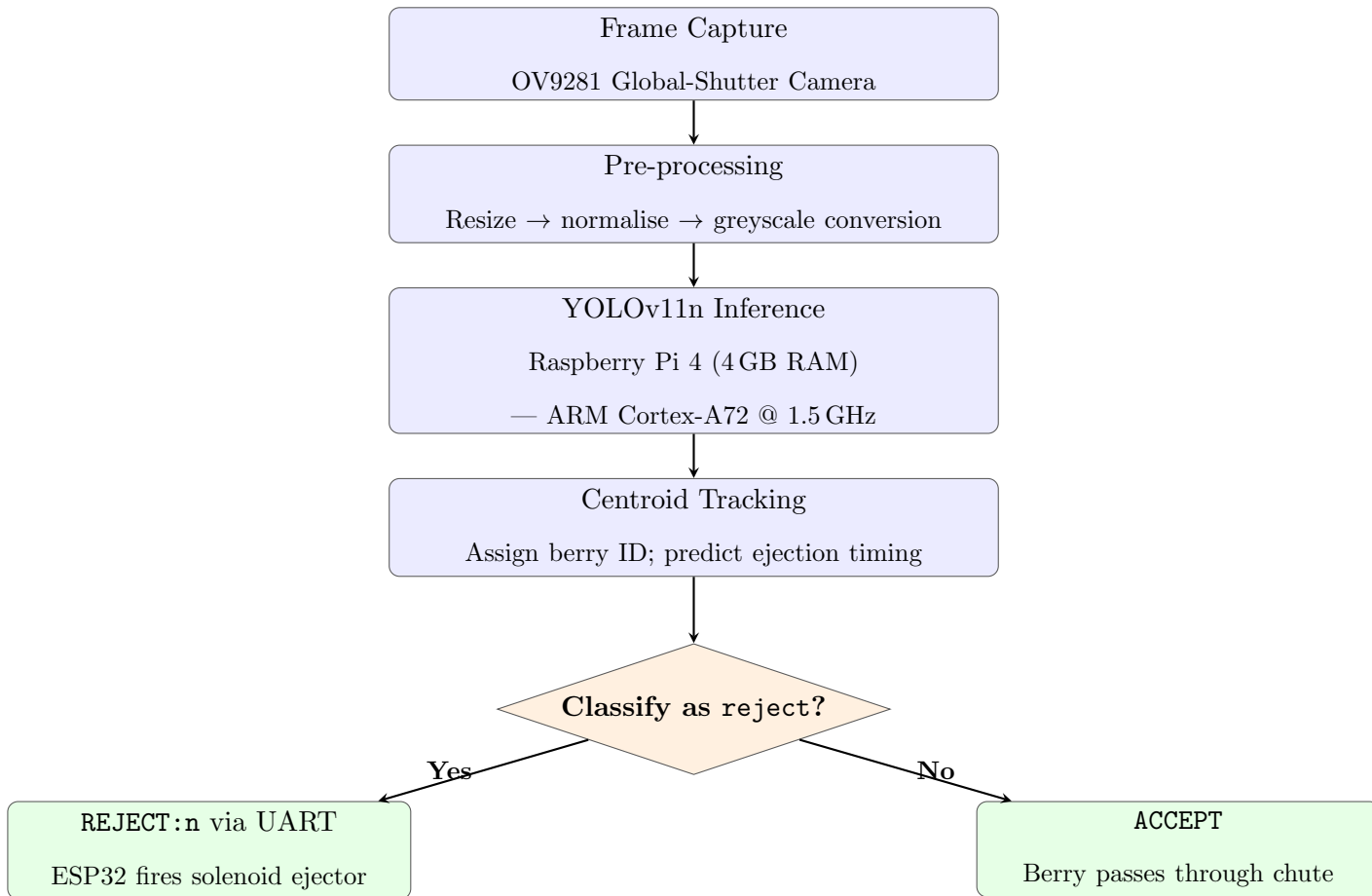


Figure 2.5: Image processing pipeline for YOLOv11-based sorter on Raspberry Pi 4: frame capture (global shutter) → pre-processing → YOLOv11 inference → classification (accept/reject) → ejection control

2.4.4 YOLOv11 on Embedded Platforms

YOLOv11, released by Ultralytics in late 2024, introduces architectural improvements to the C2f backbone and neck that improve small-object detection and reduce parameter count by $\approx 22\%$ compared to YOLOv8n [23]. Critically, it supports ONNX and TFLite export paths optimised for ARM Cortex-A72 (Raspberry Pi 4) [24], enabling inference at 25–40 fps within a 1.5 W power envelope. Its bounding-box output makes it suitable for tracking individual berries across frames [25], overcoming the whole-image classification limitation of MobileNetV2.

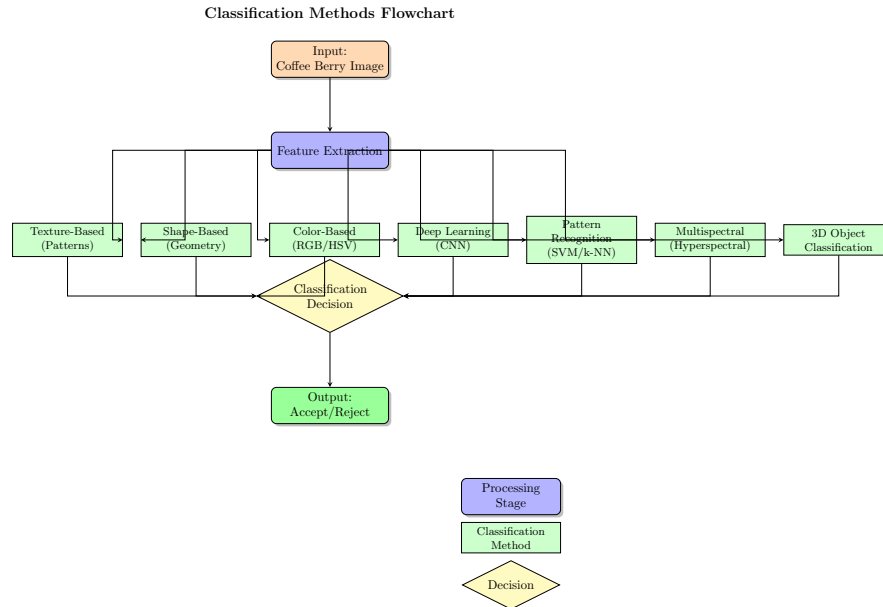


Figure 2.6: Overview of classification approaches considered: classical machine learning, CNN-based models, and single-stage detectors (YOLO family)

2.5 Binary (Accept/Reject) vs. Fine-Grained Classification

Industry coffee-sorting practice distinguishes primarily between *acceptable* (fully ripe, uniform red) and *unacceptable* (green, yellow, overripe, defective) berries [2, 7]. Semi-ripe berries—those in transitional yellow-orange states—do not meet premium export grading

standards and are classified as rejects in all major Kenyan cooperative protocols [26].

Fine-grained models that distinguish four or more intermediate classes amplify the confusion between visually similar categories (e.g., semi-ripe and ripe), as documented in the reference project [5]. Binary classification, by contrast, reduces the decision boundary to a single threshold in feature space: berries are labelled **ACCEPT** (uniform deep-red, ripe) or **REJECT** (all other states: green, yellow-orange, overripe, defective). This aligns with real-world sorting practice and substantially reduces label ambiguity in the training set [3, 27].

This project adopts the binary scheme. The YOLOv11 model is trained and evaluated on two classes only: `accept` and `reject`. Semi-ripe, overripe, unripe, and visibly defective berries all map to the `reject` class. Transfer learning from the YOLOv11n pretrained backbone [21, 28] allows the model to converge effectively on a dataset of only 2,000 images.



Figure 2.7: Visual comparison of ripe (`accept`) and unripe/semi-ripe (`reject`) coffee berries. The binary classification boundary maps to the deep-red colour transition.

2.6 Embedded Co-Processor Architectures

Offloading time-critical real-time tasks to a dedicated microcontroller has been shown to significantly improve throughput on single-board computer (SBC) vision systems [29, 30]. In multi-tier architectures, the SBC (Raspberry Pi) handles computationally heavy tasks (frame capture, neural network inference, object tracking) while a microcontroller (e.g., ESP32, STM32) is responsible for hard real-time operations: solenoid pulse timing, PWM motor control, and relay switching [27].

The **ESP32-WROOM-32** [29] is a dual-core 240 MHz Xtensa LX6 microcontroller with native UART, SPI, I²C, and 16 configurable PWM channels. Its deterministic ISR latency ($<5\ \mu\text{s}$) and hardware-timer-based solenoid firing eliminate the scheduling jitter inherent in Linux-based Pi GPIO control. This is essential for accurate berry ejection timing at berry supply speeds above $0.3\ \text{m s}^{-1}$.

The proposed co-processing architecture assigns the Pi 4 to neural-network inference and tracking, while the ESP32 exclusively handles solenoid, motor, and compressor control. The two devices communicate over a UART link at 115 200 baud, with the Pi transmitting short ASCII commands (e.g., `REJECT:2`) upon each ejection decision.

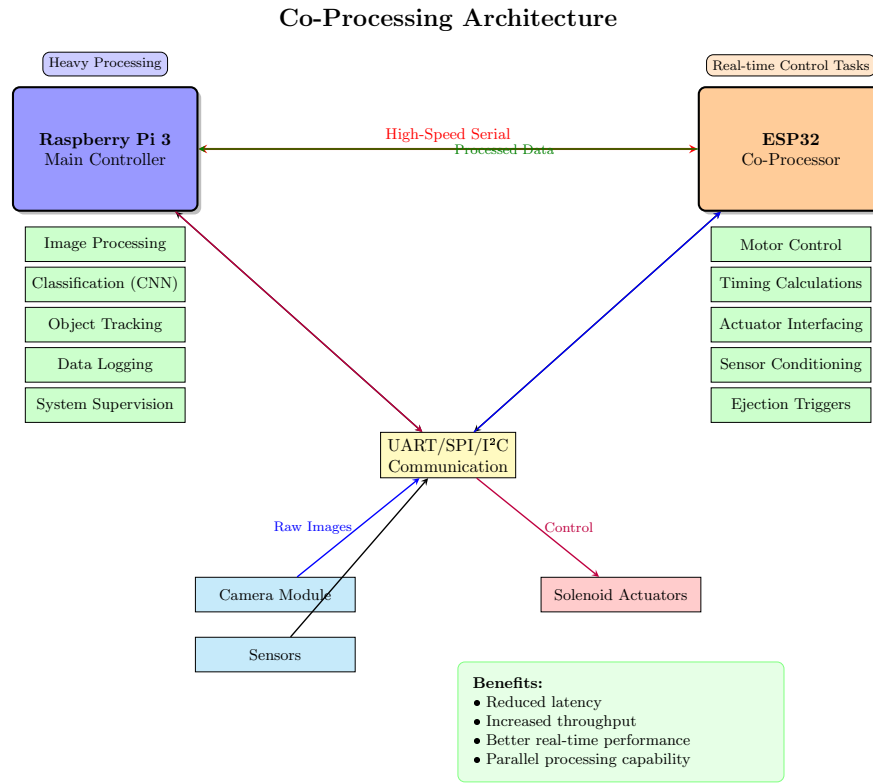


Figure 2.8: Proposed co-processing architecture: Raspberry Pi 4 handles inference; ESP32-WROOM-32 manages all hard real-time actuation via UART [24, 29].

2.7 Previous Works

The foundation of this work is a fifth-year mechatronics engineering project (FYP 22-22) at JKUAT that designed and built an automated coffee berry sorting machine [5]. That project established the core mechanical and electronic architecture on which the current work builds, and its documented performance shortcomings directly motivate the improvements proposed here.

2.7.1 System Architecture of the Reference Machine

The reference system integrated four main subsystems:

1. **Mechanical handling:** A vibrating eccentric-motor tray singulated berries from a hopper onto an inclined chute. Gravity and vibration aligned berries into a near-single-file stream before the imaging zone.
2. **Image acquisition:** A Raspberry Pi Camera Module V2 (rolling-shutter, 8 MP Sony IMX219) mounted above the chute captured frames at fixed intervals.
3. **Classification model:** The reference project initially deployed a MobileNetV2-based model [16, 17] for per-frame ripeness classification, exporting it as a TensorFlow Lite INT8 quantised file. In a subsequent revision, the team retrained using **YOLOv11n** [23] to gain bounding-box localisation per berry. Despite the model upgrade to YOLOv11n, the pipeline remained serialised on a single Raspberry Pi 3B+, so the inference-latency bottleneck persisted.
4. **Actuation:** Compressed-air solenoid jets, triggered through the Pi's GPIO pins via PN2222A BJT drivers, deflected reject-class berries into separate chutes; accepted berries continued to the accept bin under gravity.

The mechanical chute and vibrating feeder proved robust and reliable in field testing [5]. This project retains that mechanical subsystem unchanged, focusing improvements exclusively on the electronics, vision pipeline, and software layers.

2.7.2 Identified Limitations of the Reference Machine

Post-build evaluation of the reference machine identified three critical performance bottlenecks [5]:

1. **Motion blur:** The rolling-shutter sensor read out rows sequentially over a ~ 30 ms window. At operating chute speeds of $0.4\text{--}0.6\text{ ms}^{-1}$, berries translated 12–18 mm per frame, producing visible smearing that degraded classification confidence, particularly at the semi-ripe/ripe colour boundary [4].

2. **Processing latency:** Running YOLOv11n inference, GPIO triggering, and camera I/O on the Raspberry Pi 3B+ in a serialised single-threaded pipeline produced end-to-end latency of 280–400 ms per berry, limiting practical throughput to approximately 35 berries per minute—far below the commercial target of >150 per minute [5].
3. **Fine-grained class confusion:** The four-class model (unripe, semi-ripe, ripe, overripe) struggled with the visually similar semi-ripe and ripe classes. This confusion produced mis-ejections: commercially acceptable red berries were rejected and semi-ripe berries accepted into the output stream, reducing economic yield [2, 3].

Figures 2.9 and 2.10 illustrate the performance characteristics of the reference YOLOv11n model operating in four-class mode, showing the persistent classification confusion across categories.



Figure 2.9: Reference model validation predictions showing four-class classification (unripe, semi-ripe, ripe, overripe) with visible confusion between visually similar categories

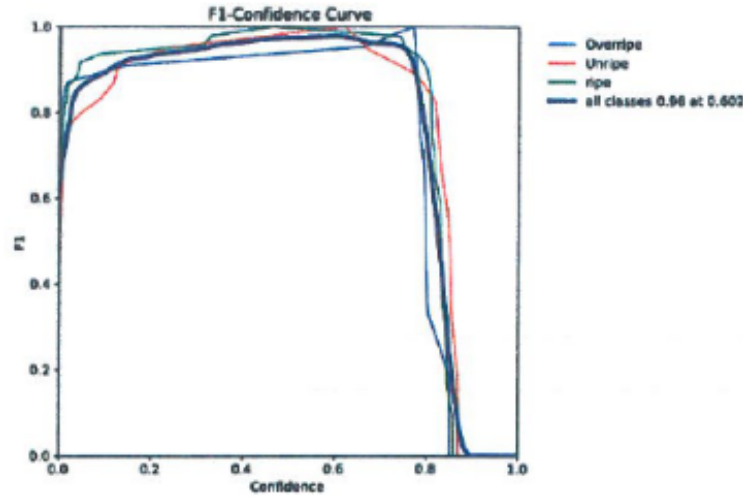


Figure 4.13: F1 score

Figure 2.10: Reference model F1-score performance across the four ripeness categories, highlighting the classification difficulty at the semi-ripe/ripe boundary

2.8 Summary of Literature Gaps Addressed by This Project

Table 2.1 summarises four specific technical gaps identified in the literature and the previous project, and maps each to the corresponding improvement proposed in this work.

Table 2.1: Literature gaps and corresponding improvements in this project

Gap identified	Root cause	This project's solution
Motion blur at operating speed (reference machine)	Rolling-shutter sensor with ~ 30 ms readout smears fast-moving berries	Replace with global-shutter Arducam OV9281 module; < 1 ms full-frame exposure eliminates motion artefacts
High end-to-end latency (280–400 ms); throughput limited to ~ 35 berries/min	Pi 4 runs inference, GPIO, and camera I/O in a serialised pipeline	Pi 4 + ESP32 co-processor [29]: Pi handles inference; ESP32 handles hard real-time solenoid timing; target < 100 ms
Limited dataset diversity and poor generalisation to new farms	Training data captured under narrow lighting and variety conditions; model overfits to a single domain	Aggregate multi-source datasets (Roboflow) and apply augmentation to improve robustness across lighting and berry varieties
Fine-grained class confusion (semi-ripe/ripe boundary) reduces economic yield	Four-class model amplifies label ambiguity at visually similar boundaries	Binary ACCEPT/REJECT scheme aligned with industry practice; YOLOv11 trained on two classes eliminates boundary confusion

3 Methodology

This chapter outlines the approach taken to improve upon the existing coffee berry sorting machine. The work spans three areas: a performance audit of the original system, targeted improvements to the sorting mechanism, and evaluation of upgraded processing hardware with a retrained classification model. Figure 3.1 shows the overall architecture of the improved system.

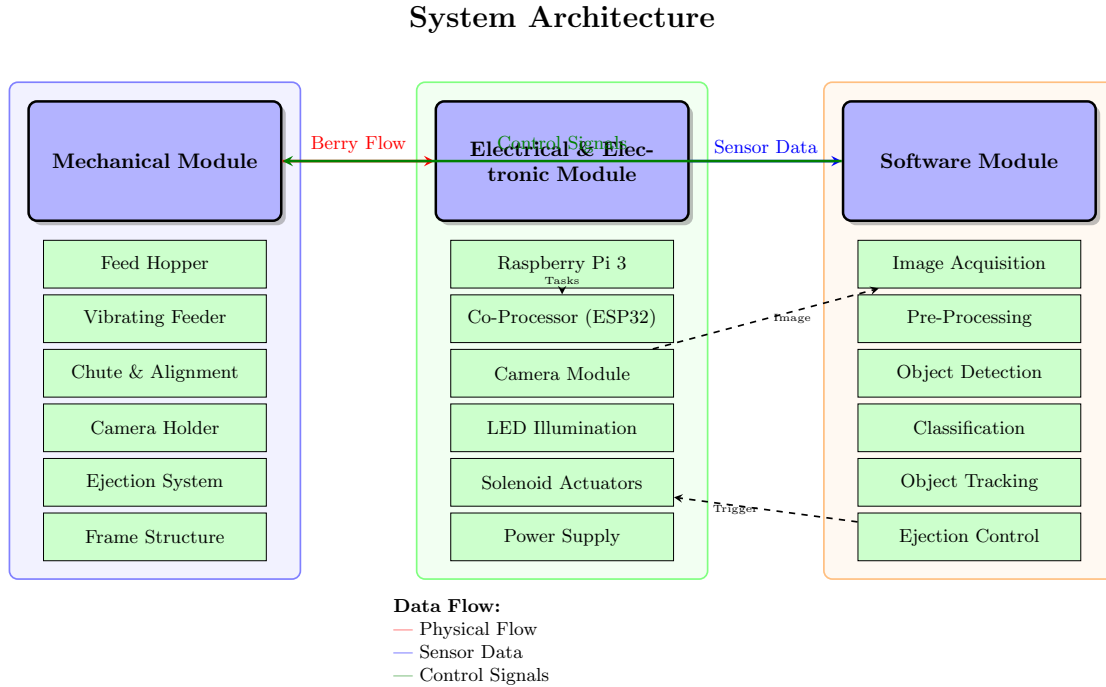


Figure 3.1: Overall architecture of the improved coffee berry sorting machine: global-shutter camera, Raspberry Pi 4 inference engine, ESP32 real-time co-processor, and MOS-FET solenoid driver stages [24, 29]

3.1 Performance Analysis of Existing Machine

A structured performance audit of the existing sorting machine was conducted to quantify the root causes of its three documented limitations: motion blur, processing latency, and

classification accuracy. The audit included:

- Measuring end-to-end pipeline latency (frame capture $\rightarrow$ inference $\rightarrow$ actuation trigger) under realistic operating conditions.
- Evaluating classification accuracy on a held-out set of berry images collected directly from the machine at operating speed, where motion blur was most evident.
- Characterising the rolling-shutter distortion of the original Raspberry Pi Camera V2 at chute speeds of 0.4–0.6 m s⁻¹.
- Identifying the contribution of the Raspberry Pi 4 running a serialised single-threaded pipeline to throughput bottlenecks.

Measured outcomes from the audit are summarised below:

- Mean end-to-end latency: ≈ 380 ms (frame capture $\rightarrow$ actuation trigger).
- Baseline classification accuracy: $\approx 72\%$ on in-motion images.
- Throughput: ≈ 35 berries per minute at 0.4–0.6 m s⁻¹ chute speed.

These findings directly informed the improvements described in the following sections.

3.2 Mechanical Subsystem — Retained and Fabricated

The mechanical subsystem of the coffee berry sorting machine was originally designed and fabricated by a previous team of fifth-year mechatronic engineering students at JKUAT [5]. Their work established the structural framework, berry feeding mechanism, chute, and ejection arrangement that form the physical backbone of the sorting system. This project acknowledges and builds upon that foundational design work. Since the mechanical components are being re-fabricated for the improved system, the design specifications from the original project have been reviewed, retained, and re-used as the basis for fabrication.

3.2.1 Berry Feeding Mechanism

Berries are loaded into a hopper and fed onto the inspection chute via a **vibrating feeder trough**. An eccentric-mass DC vibrating motor, mounted beneath the trough, imparts periodic vibration that causes berries to advance along the trough in a singulated stream toward the imaging and ejection zone. The vibration frequency and amplitude are controlled by PWM from the ESP32, allowing feed rate to be adjusted. Figure 3.2 and Figure 3.3 show the hopper and feeder trough respectively.

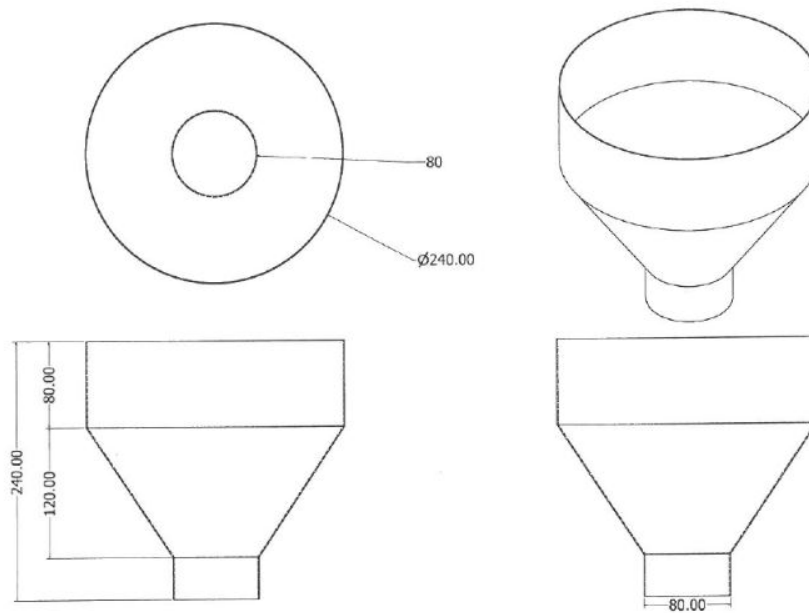


Figure 3.2: Hopper assembly — berry loading and gravity feed into the trough

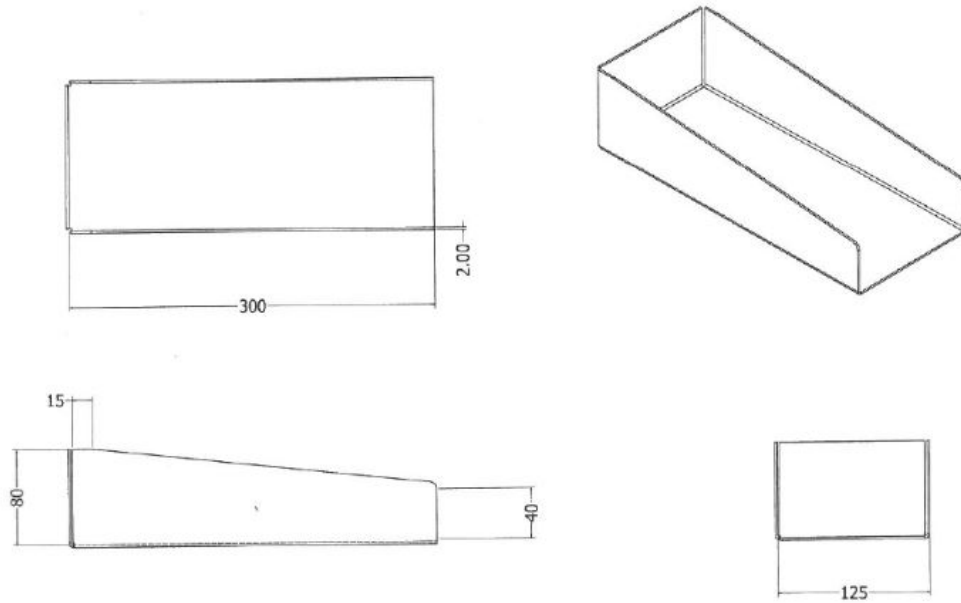


Figure 3.3: Vibrating feeder trough — singulated berry stream to inspection zone

3.2.2 Inspection Chute and Ejection

From the feeder trough, berries transit through the **inspection chute** where the camera captures frames for classification. Berries classified as **reject** are diverted by **pneumatic air ejector nozzles** mounted in the ejector holder; a short burst of compressed air deflects the berry into the reject bin. Berries classified as **accept** continue to the accept bin unimpeded. Figure 3.4 shows the chute assembly and Figure 3.5 shows the full assembled structure.

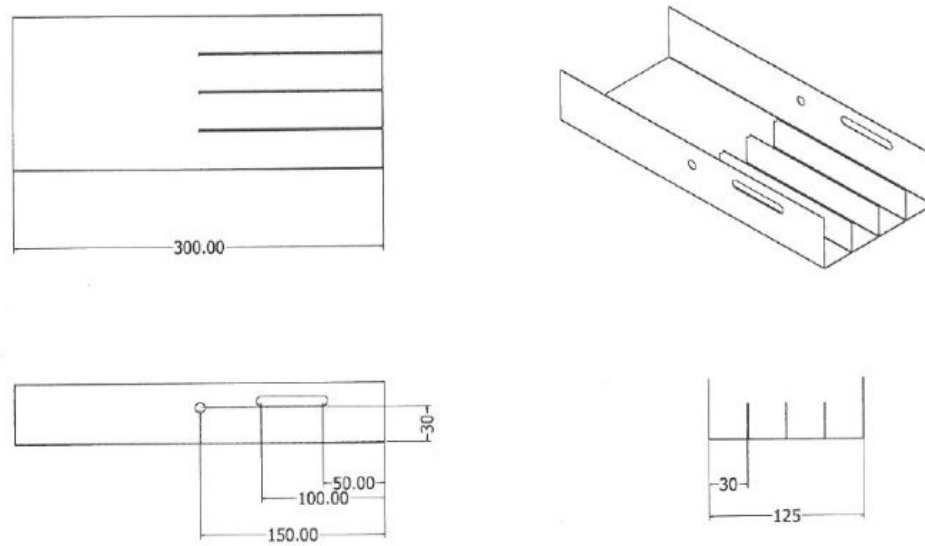


Figure 3.4: Inspection chute — berry transit path from feeder to ejection zone

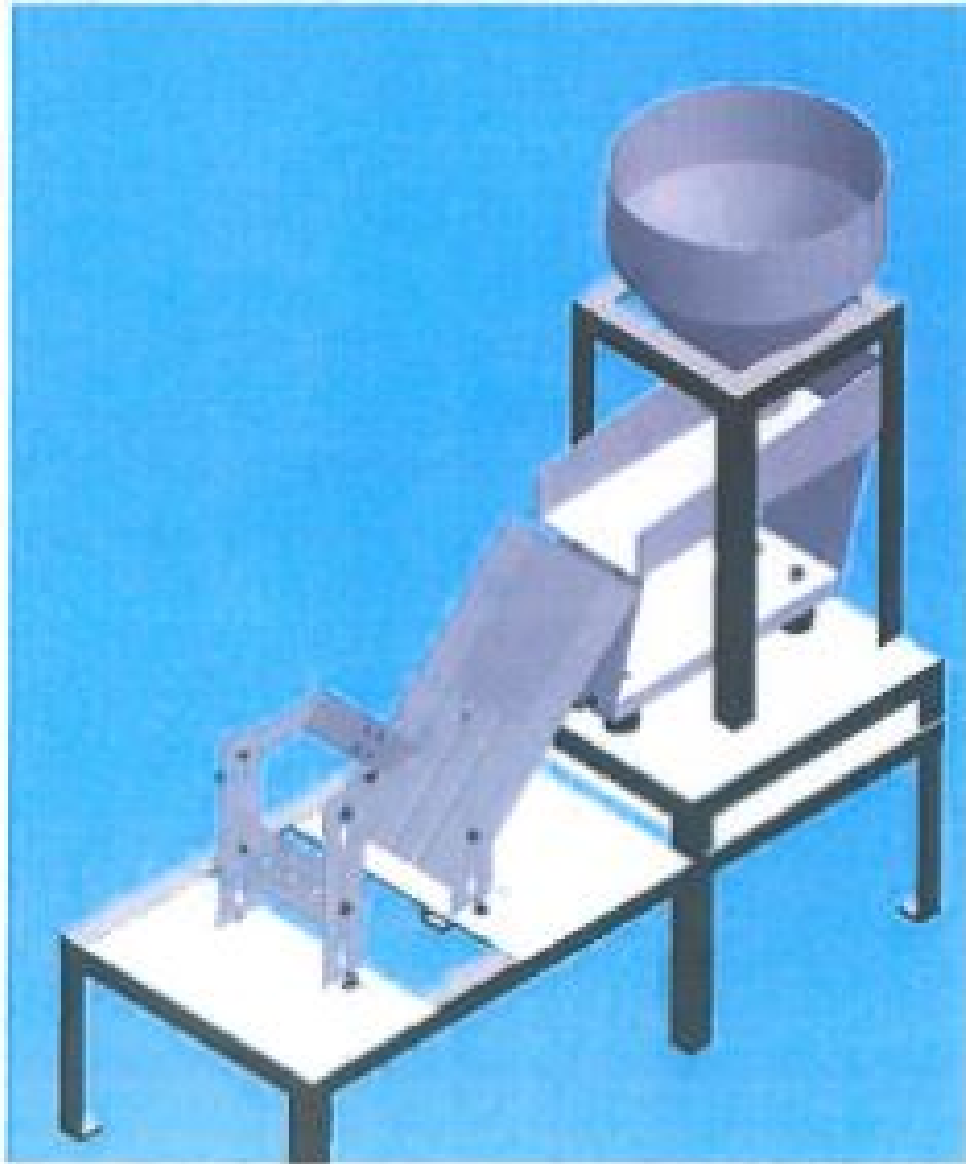


Figure 3.5: Full assembly of the sorting machine

3.2.3 Structural Frame and Components

The main structural frame is fabricated from mild steel structural sections, providing rigidity and vibration resistance. Key fabricated components include the main frame,

base plate, chute bracket, and the camera and ejector holder stand — all of which position the sensing and actuation components relative to the berry stream. These components are being re-fabricated for the improved machine using the original design drawings (see Appendix D).

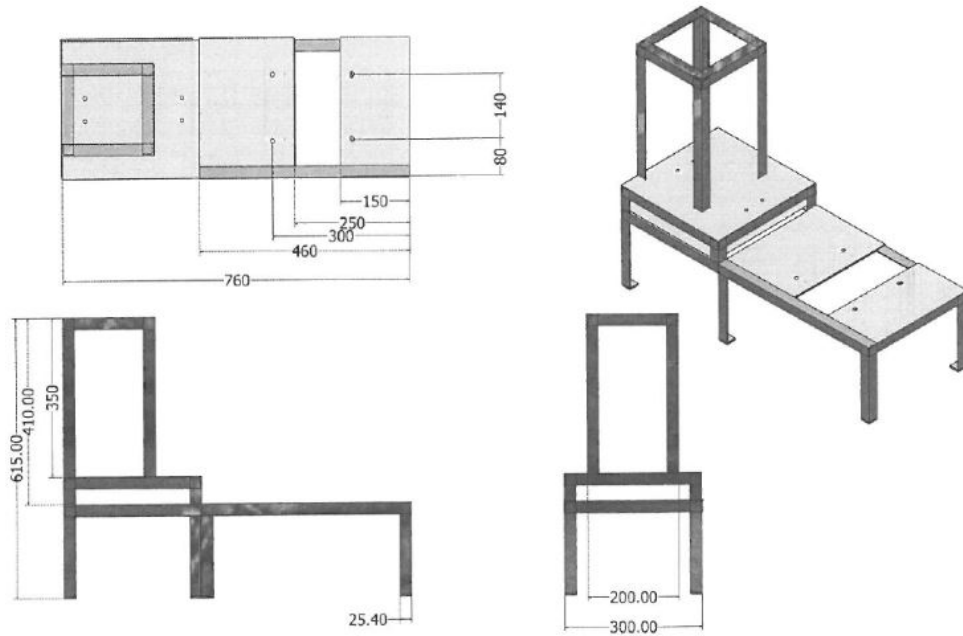
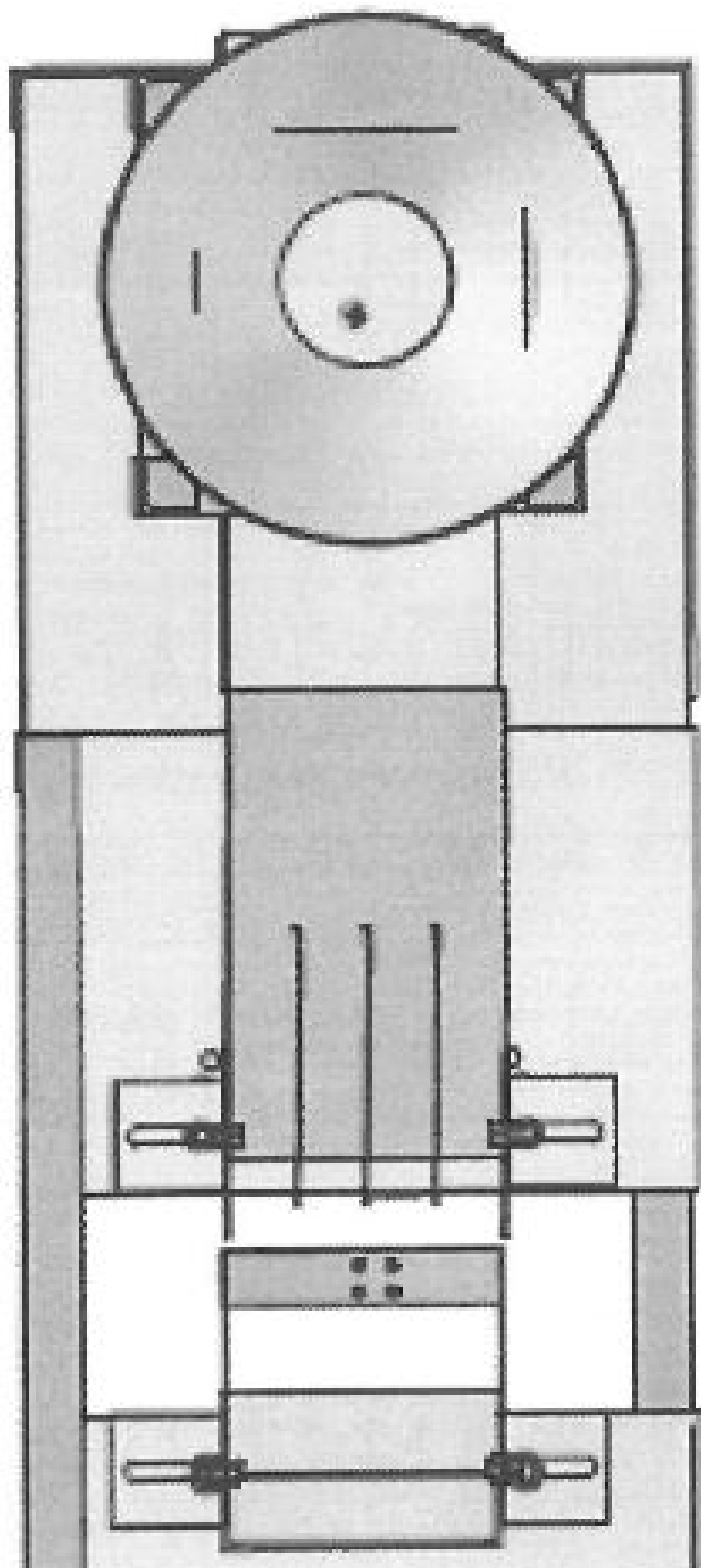


Figure 3.6: Main structural frame of the sorting machine



3.3 Dataset Collection and Model Training

The classification model was rebuilt from the ground up using upgraded hardware and a purpose-built binary dataset. The **Raspberry Pi 4 (4 GB RAM)** was selected as the inference platform, providing a quad-core Cortex-A72 at 1.5 GHz and approximately 3× the memory bandwidth of earlier Pi generations, allowing inference to run concurrently with frame capture. The MobileNetV2 TFLite classifier was replaced with **YOLOv11n**, retrained on a new binary dataset that consolidates the original four ambiguous ripeness classes into two well-separated outcomes.

MobileNetV2 presented three practical challenges in the previous pipeline: (1) it is a whole-image classifier, so frames containing multiple berries were reduced to a single label with no localisation; (2) classification confidence degraded under motion blur at operational chute speeds; and (3) the serialised inference pipeline on the Pi limited throughput and delayed ejection timing. YOLOv11n resolves these limitations by producing per-berried bounding boxes for tracking and enabling tighter detection-to-actuation timing on the upgraded Pi 4.

3.3.1 Image Acquisition

The training dataset was assembled by aggregating five publicly available, annotated coffee berry image datasets from the Roboflow Universe platform [31–35]. A total of **2,000** representative images were selected from these combined sources, balanced across the **accept** and **reject** classes. Sourcing images from multiple independently collected datasets provides a broader diversity of imaging conditions, geographical origins, lighting environments, and berry varieties than could be achieved with a single physical capture setup, improving the generalisability of the trained model.

The five contributing datasets are summarised in Table 3.1.

Table 3.1: Image dataset sources used for model training [31–35]

Dataset	Source (2023)	Roboflow URL
Coffee Cherry Ripeness Detection	Roboflow Universe	https://universe.roboflow.com/lyka-grace-tabiololo-9kido/coffee-cherry-ripeness-detection
Coffee Cherry 0ig00	Tabiolo, L. G.	https://universe.roboflow.com/lyka-grace-tabiololo-9kido/coffee-cherry-0ig00
Coffee App — Cherry Classification	Roboflow Universe	https://universe.roboflow.com/lyka-grace-tabiololo-9kido/coffee-cherry-classification/coffee-app
Coffee Cherry Classification	Roboflow Universe	https://universe.roboflow.com/lyka-grace-tabiololo-9kido/coffee-cherry-classification/coffee-cherry
Coffee Berry Detection (xhx7a)	Robopulse	https://universe.roboflow.com/robopulse/coffee-berry-xhx7a

All selected images were rescaled and centre-cropped to 640×640 pixels to match the YOLOv11 input specification.

3.3.2 Annotation

All images were annotated with bounding boxes and class labels using the Roboflow annotation platform. A **binary** classification scheme is adopted (see Section 2.5): berries are labelled as either **accept** (commercially ripe, uniform deep-red) or **reject** (all other states: unripe green, semi-ripe yellow/orange, overripe dark/black, physically damaged). This aligns with Kenyan cooperative grading standards and eliminates the semi-ripe/ripe boundary confusion that degraded the reference model.

Table 3.2: Binary coffee berry classification scheme

Class ID	Label	Action	Berry states included
0	accept	Pass to accept bin	Uniform deep-red ripe berries
1	reject	Fire ejector jet	Unripe (green), semi-ripe (yellow/orange), overripe (dark/black), and damaged berries

3.3.3 Dataset Split

The annotated dataset was split 70% train / 15% validation / 15% test using the `split_dataset.py` utility developed as part of this project.

3.4 YOLOv11 Model Training

3.4.1 Framework and Backbone

The Ultralytics YOLOv11 Nano (`yolo11n.pt`) pre-trained on COCO was selected as the backbone. This was fine-tuned using transfer learning on the coffee berry dataset.

3.4.2 Training Configuration

Table 3.3: YOLOv11 training hyperparameters (Ultralytics default training configuration) [23]

Parameter	Value
Epochs	15
Image size	640×640
Batch size	16
Optimizer	SGD (default Ultralytics)
Learning rate	0.01
Data augmentation	Mosaic, flip, HSV jitter

3.4.3 Evaluation Metrics

Model performance was evaluated using:

- **mAP@0.5** and **mAP@0.5:0.95** on the test split.
- Per-class precision and recall.
- Inference time (ms per frame) on both the training machine and the Raspberry Pi 4.

3.5 Sorting Mechanism Improvements

Based on the performance audit findings, three targeted upgrades were designed and implemented to address the root causes of poor sorting performance:

1. **Camera replacement:** The rolling-shutter Raspberry Pi Camera V2 was replaced with the Arducam OV9281 global-shutter module to eliminate motion blur.

2. **Ejector driver upgrade:** The PN2222A BJT transistors in the solenoid driver stages were replaced with IRLZ44N logic-level MOSFETs to improve switching speed and reliability at 3.3 V ESP32 drive levels.
3. **ESP32 co-processor:** An ESP32 microcontroller was introduced between the Raspberry Pi and the actuators to take over all hard real-time tasks — solenoid timing, motor PWM, relay switching — relieving the Pi’s Linux scheduler from deterministic GPIO control.

3.5.1 Hardware Platform

The system employs a co-processing architecture. The Raspberry Pi 4 (4 GB RAM) acts as the main inference engine handling image acquisition, YOLOv11 inference, and object tracking. The ESP32 microcontroller handles time-critical real-time tasks: solenoid trigger timing, motor PWM, and compressor control. Communication between the two processors is over UART at 115 200 baud.

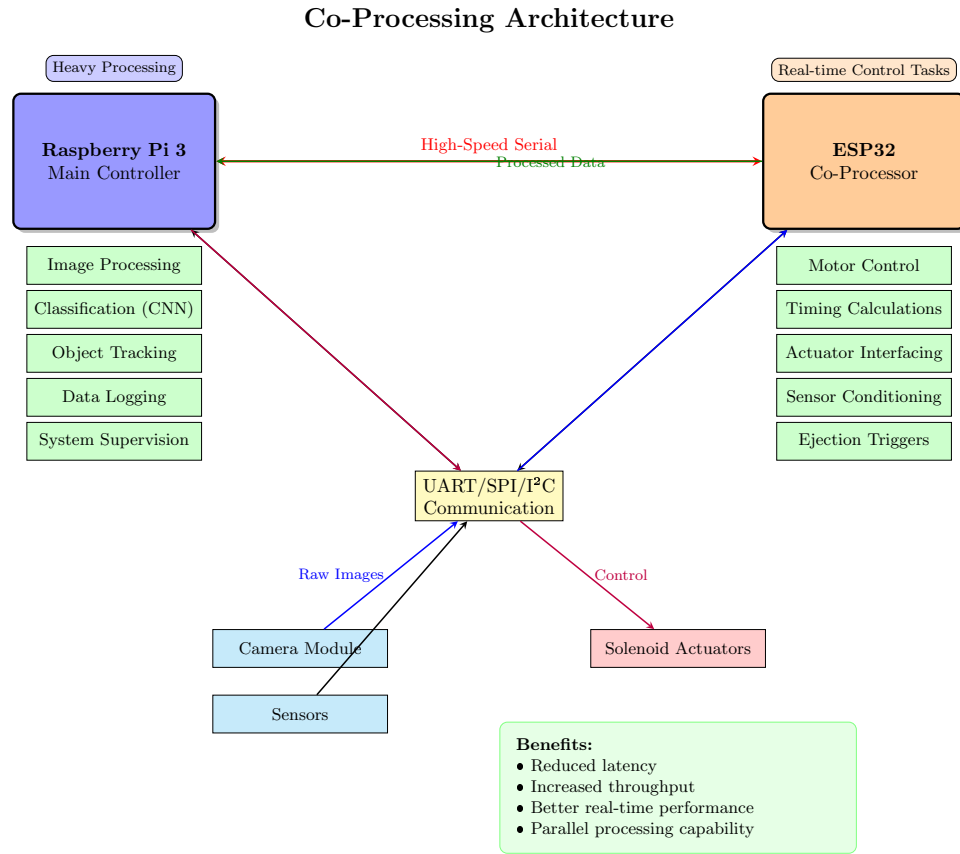
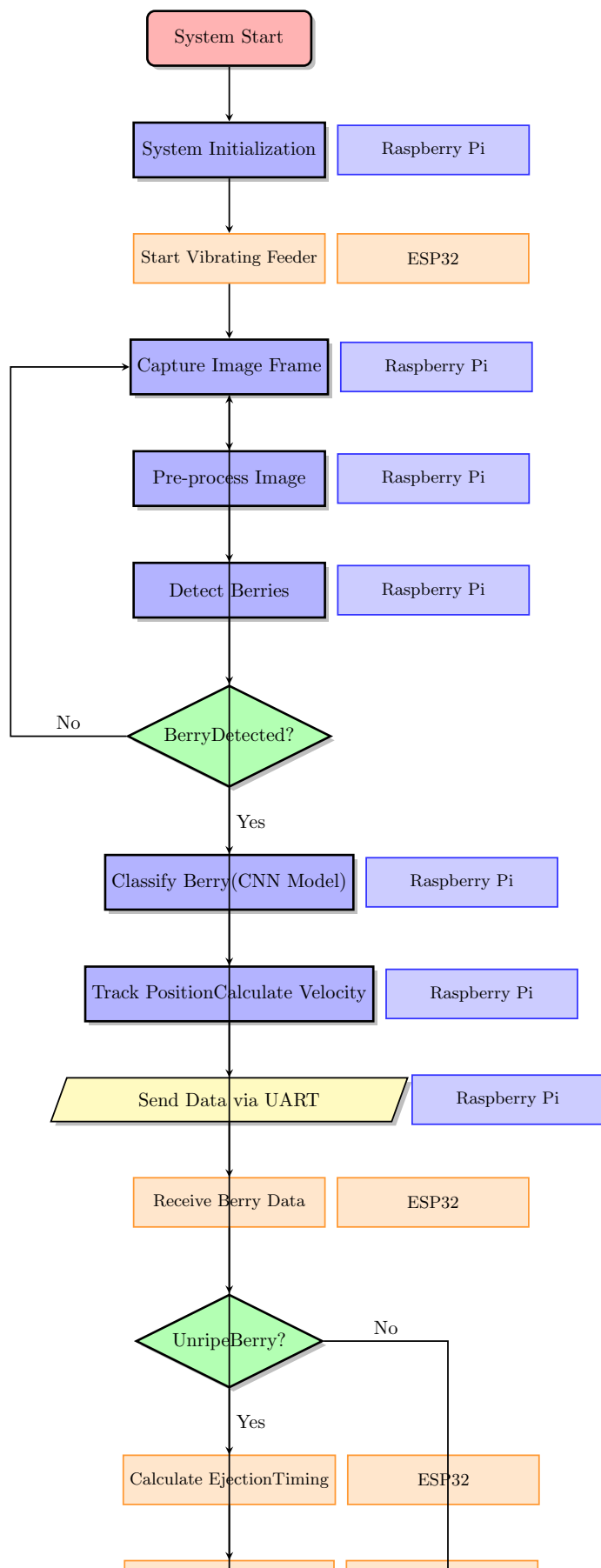


Figure 3.8: Co-processing architecture: Raspberry Pi 4 handles image processing and YOLOv11 classification; ESP32 handles motor control, actuator interfacing, and timing [24, 29]



3.5.2 Detection-to-Actuation Pipeline

The pipeline operates as follows:

1. Pi Camera captures a frame from the detection zone.
2. YOLOv11 performs inference to detect and classify berries in the frame (Figure 3.10).
3. Detected berry positions are tracked across frames using a centroid tracker [25].
4. When a tracked berry reaches the ejection zone, the Pi sends a binary decision (ACCEPT or REJECT:*n* where *n* is the ejector index) to the ESP32 via UART (Figure 3.11).
5. ESP32 fires the corresponding solenoid jet for a calibrated pulse duration; ACCEPT commands generate no solenoid activation.

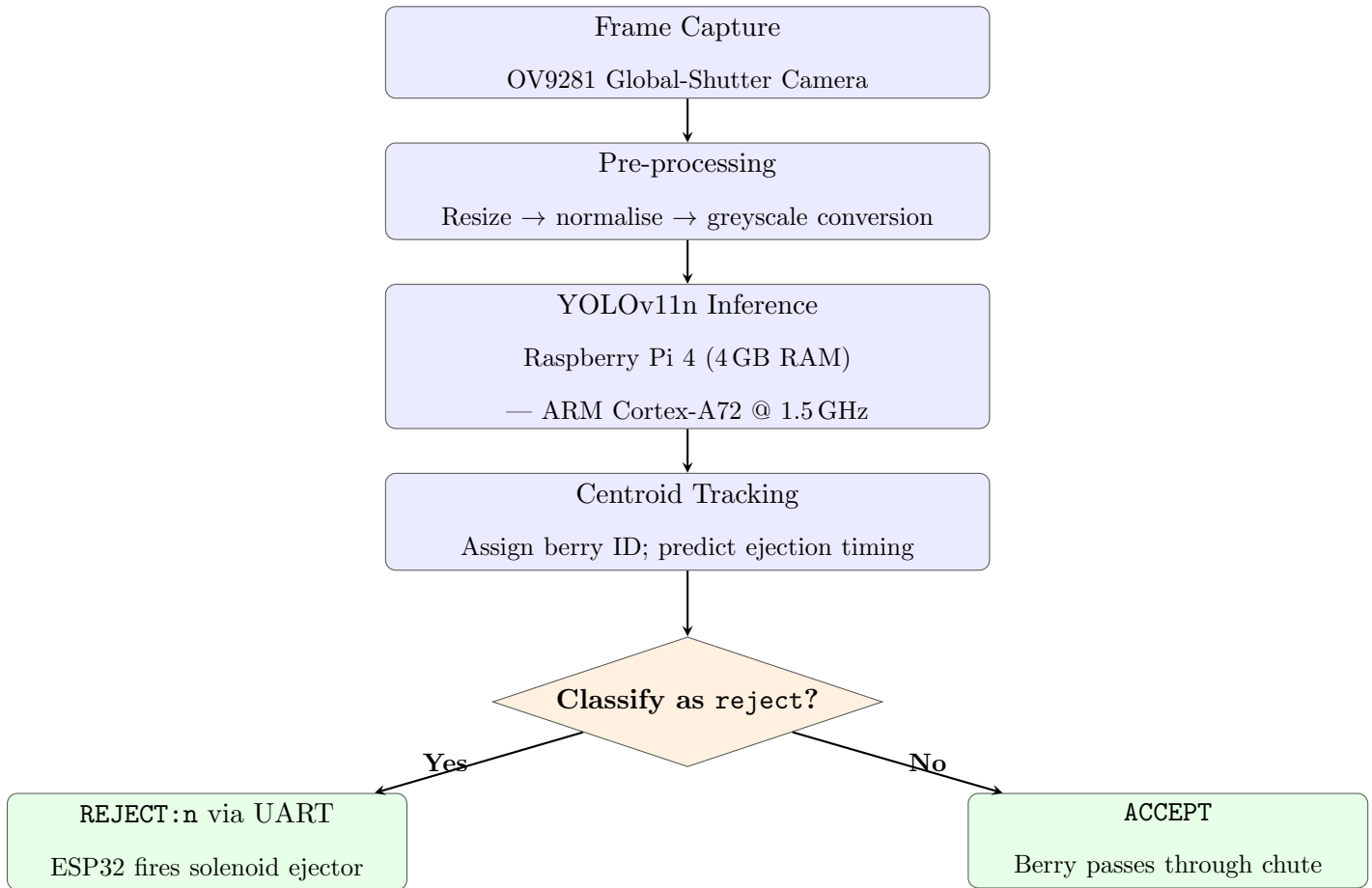


Figure 3.10: Image processing pipeline for the YOLOv11-based sorter: frame capture (global shutter) → pre-processing → YOLOv11 inference on Raspberry Pi 4 → centroid tracking → UART command to ESP32

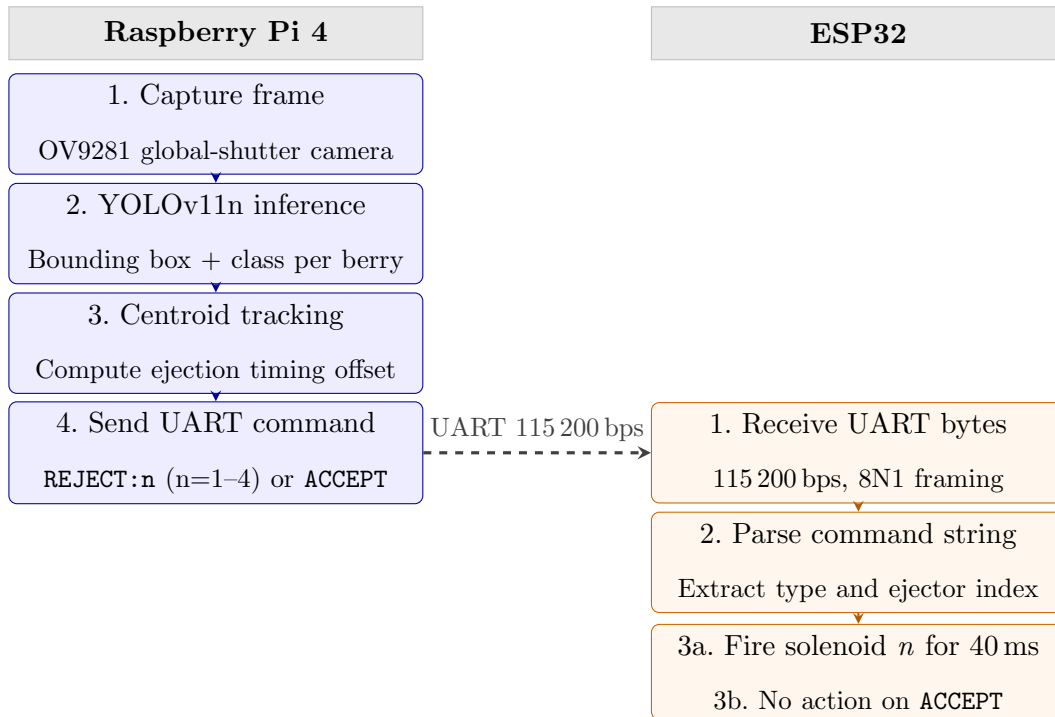


Figure 3.11: UART communication and data flow between Raspberry Pi 4 and ESP32: the Pi sends **REJECT:n** or **ACCEPT** commands after each inference decision; the ESP32 handles all real-time solenoid actuation.

3.5.3 ESP32 Pin Assignment and Wiring

Table 3.4 defines the pin assignments for the ESP32 microcontroller and its connections to the Raspberry Pi, solenoid drivers, vibrating motor, pneumatic compressor, and LED status indicator. All signal lines between the Pi and the ESP32 operate at 3.3 V logic. The 12 V solenoid and motor supplies are isolated with a common ground return tied back to all control electronics.

Table 3.4: Raspberry Pi, ESP32, and Peripheral Wiring Assignment

Function	Device	Pin/Port	Notes
UART Communication (Pi ↔ ESP32)			
UART TX	Raspberry Pi	GPIO 14 (TXD)	3.3 V; transmit to ESP32 RX
UART RX	Raspberry Pi	GPIO 15 (RXD)	3.3 V; receive from ESP32 TX
UART RX	ESP32	GPIO 16	Receive from Pi TXD
UART TX	ESP32	GPIO 17	Transmit to Pi RXD
Baud rate	Both	115 200 bps, 8N1	Serial protocol parameters
Ejector Solenoid Control (ESP32)			
Ejector 1	ESP32	GPIO 4	MOSFET gate + 220 Ω resistor
Ejector 2	ESP32	GPIO 5	Flyback diode across solenoid
Ejector 3	ESP32	GPIO 12	1N4007 (cathode to +12 V)
Ejector 4	ESP32	GPIO 13	100 k Ω pull-down for safety
Vibrating Motor (ESP32)			
Motor PWM	ESP32	GPIO 14 (PWM)	MOSFET gate driver
Motor supply	12 V rail	+12 V, GND	Flyback diode protection
Pneumatic Compressor (ESP32)			
Compressor	ESP32	GPIO 21 (relay)	Relay driver; on/off switching
Compressor supply	12 V rail	+12 V, GND	Relay coil protection diode
LED Status Indicator (ESP32)			
LED PWM	ESP32	GPIO 22 (PWM)	MOSFET gate driver
LED supply	12 V rail	+12 V, GND	Flyback diode protection
Power Supply			
Logic 5 V	Buck converter	Pi 5 V, 3 A+	Raspberry Pi 4 supply
Logic 5 V	Buck converter	ESP32 5 V, 2 A+	Onboard 3.3 V regulator
Motor/actuator 12 V	Battery/adaptor	10 A fused	Main actuator supply
Common GND	All devices	Star ground point	All GND returns tied together

Each ejector solenoid circuit includes a $220\,\Omega$ gate resistor for current limiting and a $100\,\text{k}\Omega$ pull-down to prevent false firing on power-up. A flyback diode (1N4007, cathode to $+12\,\text{V}$) across each solenoid suppresses inductive transient kickback when the MOSFET switch opens.

3.5.4 Transistor Technology Selection: BJT vs. Logic-Level MOSFET

The reference machine used bipolar junction transistors (BJTs — PN2222A / BC547) as the switching elements in the ejector solenoid driver stages. While BJTs are cheap and widely available, they impose a fundamental switching-speed penalty that becomes critical in a timer-sensitive ejection circuit.

Table 3.5: BJT (PN2222A) vs. Logic-Level MOSFET (IRLZ44N) switching comparison

Parameter	BJT (PN2222A)	MOSFET (IRLZ44N)
Control mechanism	Current-controlled (I_B)	Voltage-controlled (V_{GS})
Drive voltage from ESP32	Must source $\sim 5\,\text{mA}$ base current	Gate capacitance only; $< 1\,\text{mA}$
Turn-on threshold	$V_{BE} \approx 0.7\,\text{V}$ (forward bias)	$V_{GS(th)} = 1\text{--}2\,\text{V}$ (logic-level gate)
Storage time (turn-off delay)	$t_{stg} = 200\text{--}600\,\text{ns}$	None (no minority carrier storage)
Turn-on time t_r	$\approx 100\,\text{ns}$	$< 20\,\text{ns}$ (gate charge limited)
Turn-off time t_f	$\approx 250\,\text{ns} + \text{storage time}$	$< 50\,\text{ns}$
On-state voltage drop	$V_{CE(sat)} \approx 0.3\,\text{V}$	$V_{DS(on)} = I_D \times R_{DS(on)} \approx 50\,\text{mV}$ at $1\,\text{A}$
Maximum current (continuous)	$600\,\text{mA}$	$47\,\text{A}$
Suitability at $3.3\,\text{V}$ gate/base	Marginal (V_{BE} barely saturates)	Full enhancement; specified at $3.3\,\text{V}$

The critical advantage of the MOSFET for this application is the absence of **minority carrier storage time**. A BJT in saturation accumulates charge carriers in the base region; when the switching signal is removed these must recombine before the transistor turns off, adding $200\text{--}600\,\text{ns}$ of unavoidable delay. For a solenoid fired within a $45\,\text{ms}$ timing window at a chute speed of $0.5\,\text{m s}^{-1}$, this is negligible in isolation, but when combined with upstream inference latency any additional margin matters.

The MOSFET also operates with negligible gate drive current from the ESP32 3.3 V rail, whereas the BJT requires the GPIO to source ≈ 5 mA of base current. With four simultaneous ejectors the BJT design loads the microcontroller's output stage; the MOSFET design imposes essentially zero steady-state GPIO load.

For these reasons the IRLZ44N logic-level MOSFET replaces the PN2222A BJT across all four ejector channels, the motor driver, and the LED strip driver.

3.5.5 Solenoid Driver Circuit

Each of the four ejector channels uses an identical N-channel MOSFET stage. The ESP32 GPIO output (3.3 V logic) drives the IRLZ44N gate through a $220\ \Omega$ series resistor, which limits gate-charge current and damps any parasitic resonance with the gate capacitance. A $100\ \text{k}\Omega$ pull-down resistor between gate and source holds the MOSFET firmly off during ESP32 power-up and reset, preventing false solenoid firing. The solenoid coil (12 V, $\approx 22\ \Omega$ DC resistance) is connected between the drain and the 12 V supply rail. A 1N5819 Schottky flyback diode, cathode to +12 V and anode to the drain, clamps the inductive back-EMF spike to within the diode's forward voltage (0.3–0.5 V) above the rail when the MOSFET turns off. The source is tied directly to the common ground reference that connects the 12 V actuator supply and the ESP32 logic supply, ensuring no ground-loop offset between the control and power sections. This circuit topology corresponds directly to the design verified in Proteus and tabulated in Table 3.4.

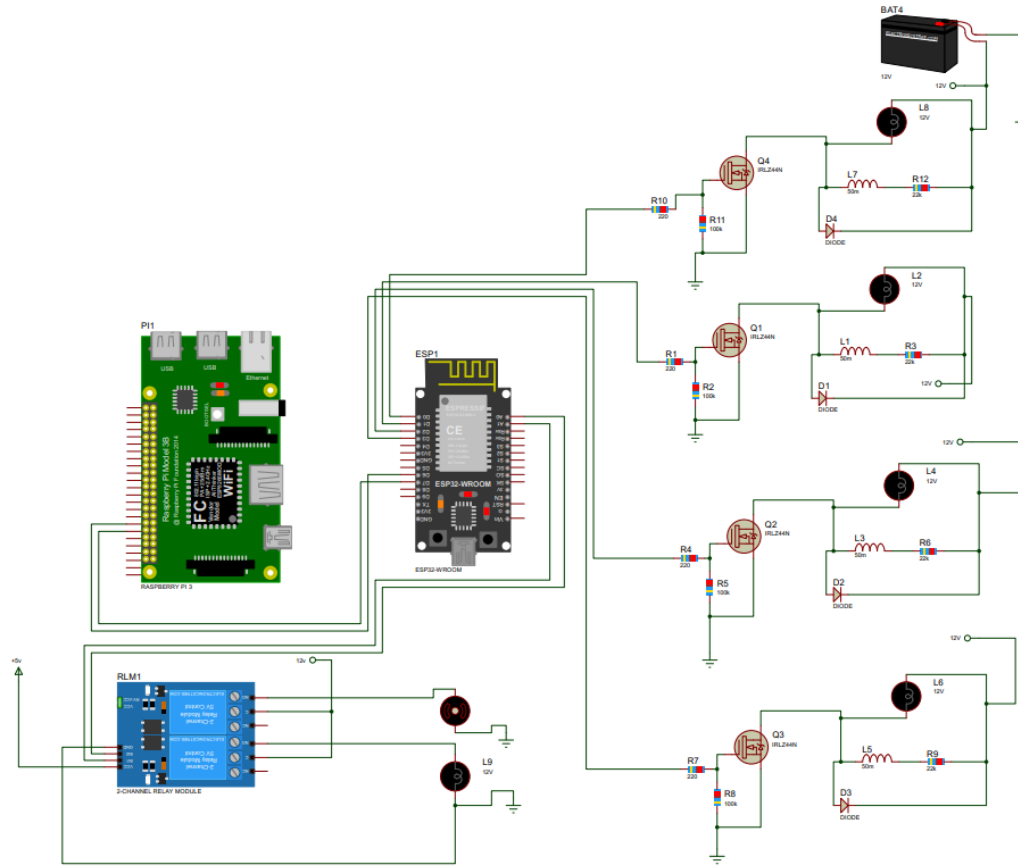


Figure 3.12: Fritzing circuit diagram of the ESP32 solenoid driver implementation: IRLZ44N MOSFET switching stages, flyback diodes, gate resistors, and Raspberry Pi UART connection

3.6 Circuit Simulation in Proteus

Prior to physical breadboard construction, the ESP32-based control circuit was designed and verified using **Proteus ISIS** (Labcenter Electronics). Proteus supports simulation of the ESP32 microcontroller alongside IRLZ44N MOSFET drivers, relay circuits, solenoid load models, and UART virtual terminals, enabling full functional verification before any hardware was procured. Figure 4.3 in Chapter 4 shows the complete Proteus schematic.

3.6.1 Simulation Scope

The simulation covered the following subsystems:

- **UART communication:** Simulated serial exchange between a virtual UART terminal (representing the Raspberry Pi) and the ESP32. Correct baud rate, 8N1 frame format at 115 200 bps, and message parsing logic were verified using Virtual Serial Port instruments.
- **Solenoid ejector drivers:** Four N-channel MOSFET switching stages (one per ejector), each with $220\ \Omega$ gate resistor, 1N4007 flyback diode, and $100\ \text{k}\Omega$ pull-down, were simulated and switching waveforms verified against the specification in Table 3.4.
- **Vibrating motor PWM control:** GPIO 14 PWM output was simulated into a motor load model; variable duty cycle confirmed correct speed control behaviour.
- **Relay-based compressor control:** GPIO 21 relay circuit verified for correct ON/OFF sequencing and protection diode operation under simulated load.
- **LED status indicator:** GPIO 22 PWM dimming into an LED model confirmed brightness control from 0% to 100%.

3.6.2 Simulation Methodology

The ESP32 firmware was compiled (Arduino framework) and loaded into the Proteus virtual ESP32-WROOM-32 component. Input stimuli consisting of binary classification command strings (e.g., `REJECT:1`, `REJECT:3`, `ACCEPT`) were injected via a virtual serial terminal scripted to replicate Raspberry Pi UART output. Oscilloscope probes were placed on each MOSFET gate pin and solenoid node to capture switching waveforms. The simulation confirmed correct actuation of all four ejector channels, motor PWM behaviour, compressor relay switching, and UART framing before physical construction

commenced. The complete Proteus schematic and waveform captures from this simulation are presented as results in Section 4.3.

3.7 Validation Strategy

System performance will be validated through:

- **image test:** Run the YOLOv11 model on the held-out test set and report mAP@0.5, mAP@0.5:0.95, per-class precision and recall.
- **Throughput test:** Measure berries processed per minute at several belt speeds and identify the optimal operating point.
- **Latency test:** Measure end-to-end pipeline time from frame capture through YOLOv11 inference to UART solenoid trigger on the Pi 4 + ESP32 hardware.

4 Results and Discussion

4.1 Overview

This chapter presents the results and current status of work completed during the interim phase of the project. Work completed to date covers two main areas: (1) circuit design and Proteus simulation to verify the MOSFET solenoid valve driver switching behaviour, and (2) YOLOv11n model training on the assembled binary coffee berry dataset. Baseline performance measurements against the physical machine and full hardware integration are planned for the next phase.

4.2 Design Rationale

The existing sorting machine has three documented limitations: motion blur from a rolling-shutter camera at operational chute speed, high end-to-end pipeline latency from a single-threaded inference pipeline on an older Raspberry Pi platform, and classification inaccuracies at the semi-ripe/ripe boundary in the original four-class model. These limitations motivate the hardware and software improvements described in the following sections. Quantitative baseline measurements will be conducted when the improved physical system is assembled and tested.

4.2.1 Design Drawings and Assembly Outputs

Figure 4.1 presents representative mechanical design drawings used for re-fabrication, showing the frame and chute geometry with title blocks and dimensions. These drawings provide the structural reference for fabrication and assembly alignment.

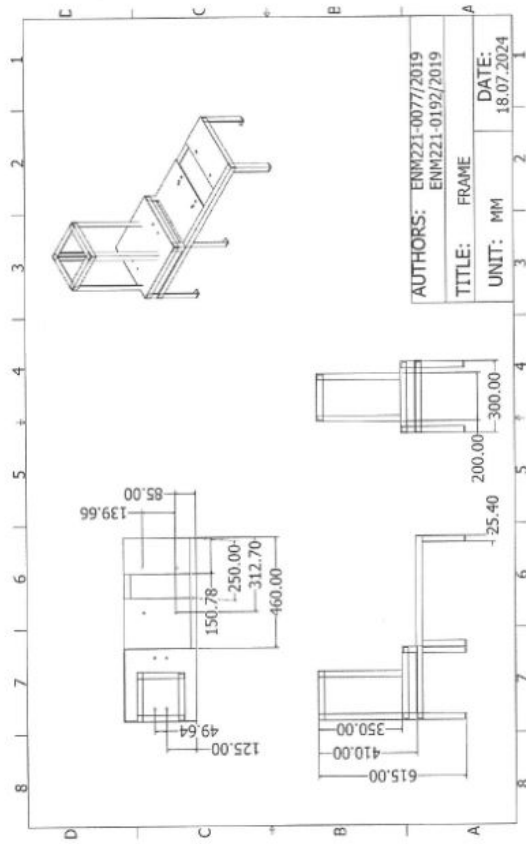
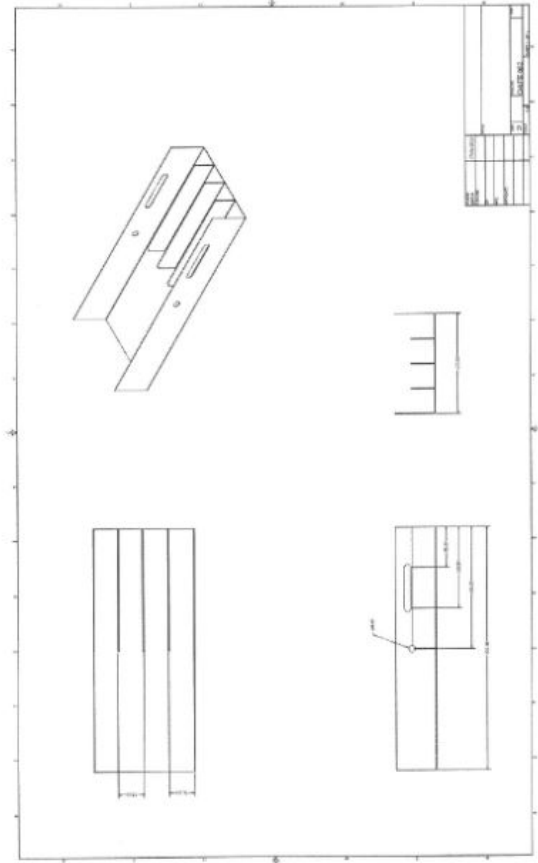


Figure 6.5: Frame

(a) Main frame design drawing



(b) Chute design drawing

Figure 4.1: Mechanical design drawings used for fabrication and assembly alignment

4.3 Sorting Mechanism Improvements

Three targeted upgrades were implemented to address the root causes identified in the performance audit:

1. **Camera replacement:** The rolling-shutter Raspberry Pi Camera V2 was replaced with the **Arducam OV9281 Global-Shutter Module** (CSI-2, 1 MP, Sony OV9281). Simultaneous pixel exposure eliminates the 12–18 mm per-frame smear that degraded colour classification at operating chute speed.

2. **Ejector driver transistor upgrade:** PN2222A BJTs were replaced with **IRLZ44N logic-level MOSFETs**. The MOSFET has no minority-carrier storage time, giving sub-20 ns turn-on versus $\approx 100 \text{ ns} + 200\text{--}600 \text{ ns}$ storage for the BJT. Full drive is achieved direct from the ESP32 3.3 V GPIO with negligible steady-state gate current.
3. **ESP32 co-processor:** An ESP32 microcontroller was introduced to handle all hard real-time tasks—solenoid pulse timing, motor PWM, relay switching—via deterministic hardware timers ($< 5 \mu\text{s}$ ISR latency), relieving the Raspberry Pi Linux scheduler from GPIO-critical paths.

4.3.1 Procured and Verified Hardware

The following components have been procured, bench-tested, and verified for compatibility:

- Raspberry Pi 4 (4 GB RAM): upgraded main inference platform.
- **Arducam OV9281 Global-Shutter Camera Module:** replacement imaging sensor.
- ESP32 DevKit: real-time actuation controller.
- LED ring light: consistent illumination above the detection zone.
- 2-channel optocoupler relay module (PC817 + SRD-05VDC): compressor and auxiliary switching.
- IRLZ44N MOSFETs ($\times 4$), 1N5819 Schottky diodes, 220Ω and $100 \text{ k}\Omega$ resistors.

The high-level system block diagram, showing the processor hierarchy, communication links, and actuator outputs, is presented in Figure 4.2.

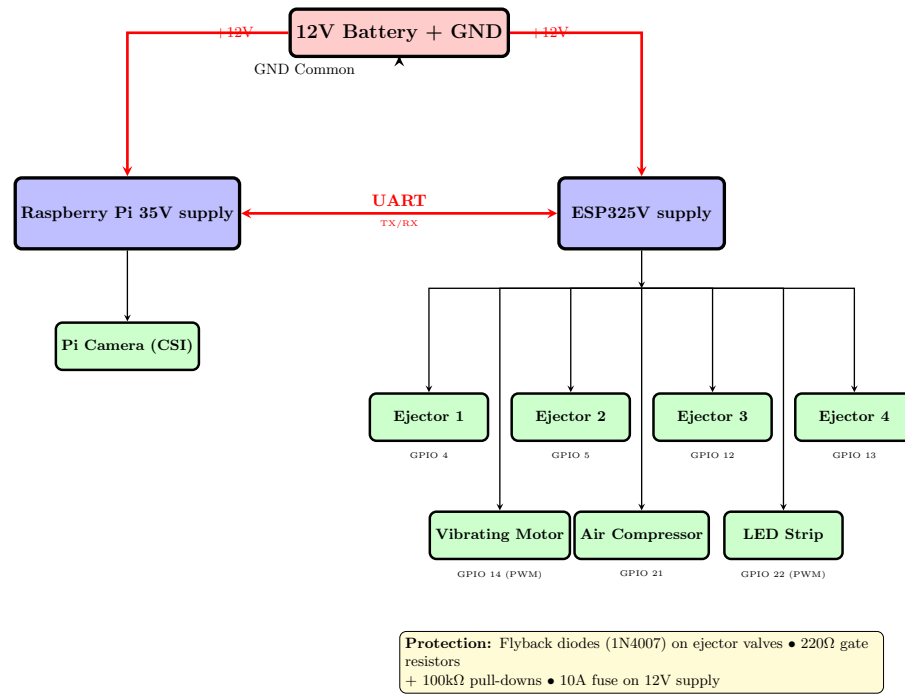


Figure 4.2: System block diagram: processor hierarchy, communication links, and actuator outputs with power distribution

4.3.2 Proteus Circuit Simulation Results

Prior to physical construction, the ESP32 control circuit was fully simulated in Proteus ISIS to verify all subsystems before hardware was assembled.

Full Circuit Overview Figure 4.3 shows the complete Proteus schematic: four IRLZ44N ejector channels, vibrating motor PWM stage, LED strip driver, 2-channel optocoupler relay module, and UART virtual terminal representing the Raspberry Pi link.

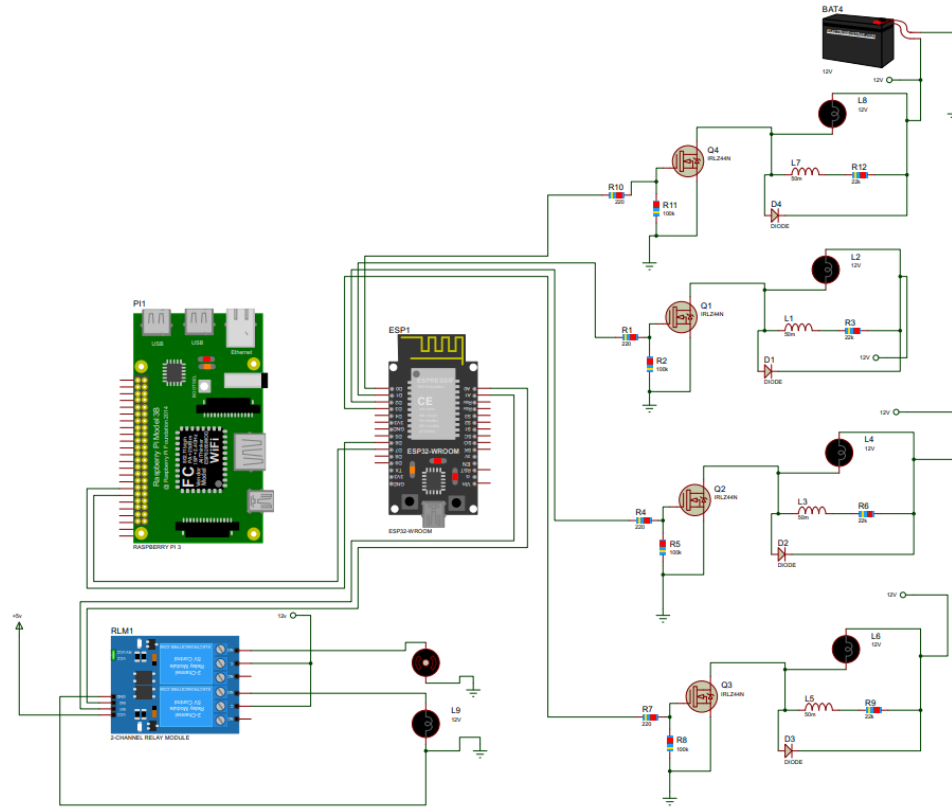


Figure 4.3: Complete Proteus ISIS simulation: ESP32 with four IRLZ44N solenoid driver channels, vibrating motor PWM, LED strip driver, 2-channel optocoupler relay, and UART virtual terminal

Solenoid Driver Switching Waveforms Figures 4.4 and 4.5 show oscilloscope captures of the MOSFET turn-on and turn-off events respectively.

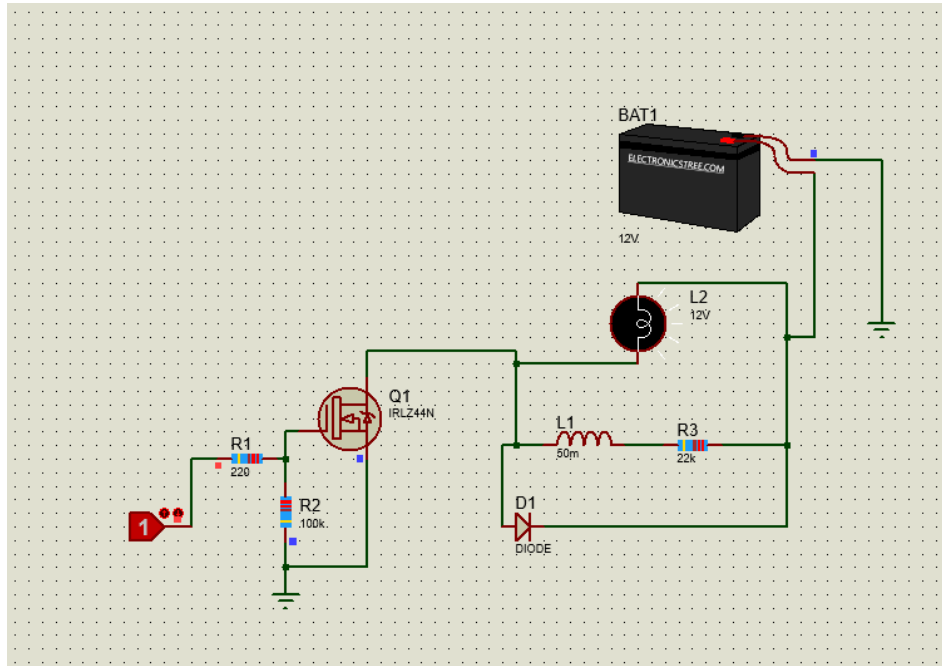


Figure 4.4: Proteus oscilloscope: MOSFET turn-ON — gate rises to 3.3 V and drain falls from 12 V to ≈ 0 V with no ringing (IRLZ44N, 220 Ω gate resistor, 100 k Ω pull-down)

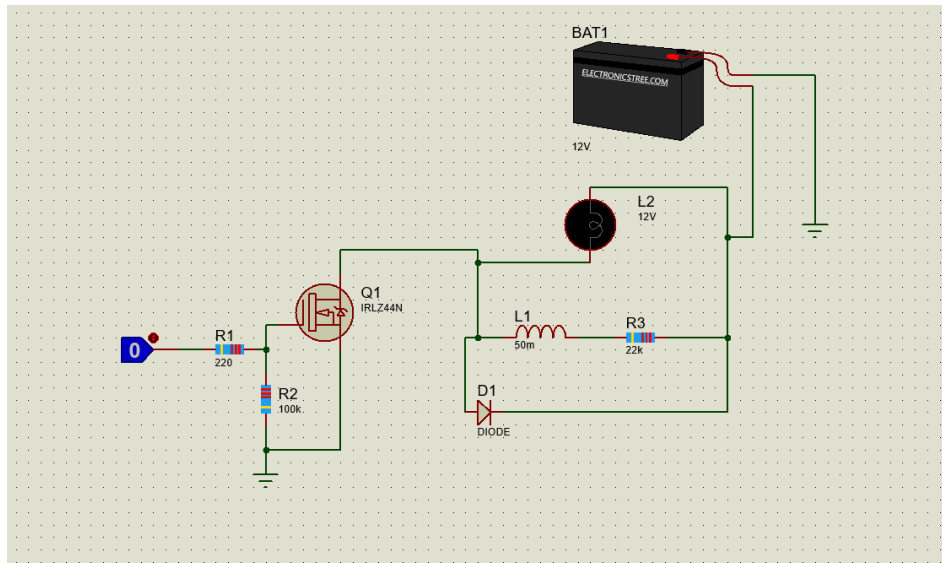


Figure 4.5: Proteus oscilloscope: MOSFET turn-OFF — drain recovers to 12 V; 1N5819 clamps inductive kickback spike within 0.5 V of the supply rail

Key results from the waveform analysis:

- All four MOSFET channels switched correctly on receipt of `REJECT:n` commands.
- No gate ringing observed; $220\ \Omega$ series resistor adequately damped parasitic oscillation.
- Flyback diode clamped solenoid turn-off spike to $+12.5\ \text{V}$ ($0.5\ \text{V}$ above rail).
- $100\ \text{k}\Omega$ pull-down confirmed zero false firing during ESP32 power-on reset.

4.4 Model Training Results

The classification model was built using **YOLOv11n** (Ultralytics), trained on a 2,000-image binary dataset assembled from five publicly available Roboflow datasets (detailed in Section 3.3). The selected inference platform is the **Raspberry Pi 4 (4 GB RAM)** [24], which provides $\approx 3\times$ the memory bandwidth of earlier Pi generations, enabling YOLOv11 inference to overlap with the next frame capture.

4.4.1 Dataset and Class Reduction

The dataset comprises **2,000 images** drawn from five Roboflow Universe coffee berry datasets [31–35] and annotated with two labels: **accept** (fully ripe, deep-red) and **reject** (all other states: unripe, semi-ripe, overripe, damaged). Consolidating four ambiguous ripeness classes into two well-separated outcomes eliminates the dominant misclassification boundary observed in the reference model.

Table 4.1: Dataset split summary (2,000-image binary dataset)

Split	Images	accept instances	reject instances
Train	1,400	TODO	TODO
Validation	300	TODO	TODO
Test	300	TODO	TODO
Total	2,000	TODO	TODO

4.4.2 YOLOv11n Training Results

The YOLOv11n model was trained for 15 epochs on the binary coffee berry dataset. Figure 4.6 shows the training results including loss curves, precision, recall, and mAP metrics across all epochs.

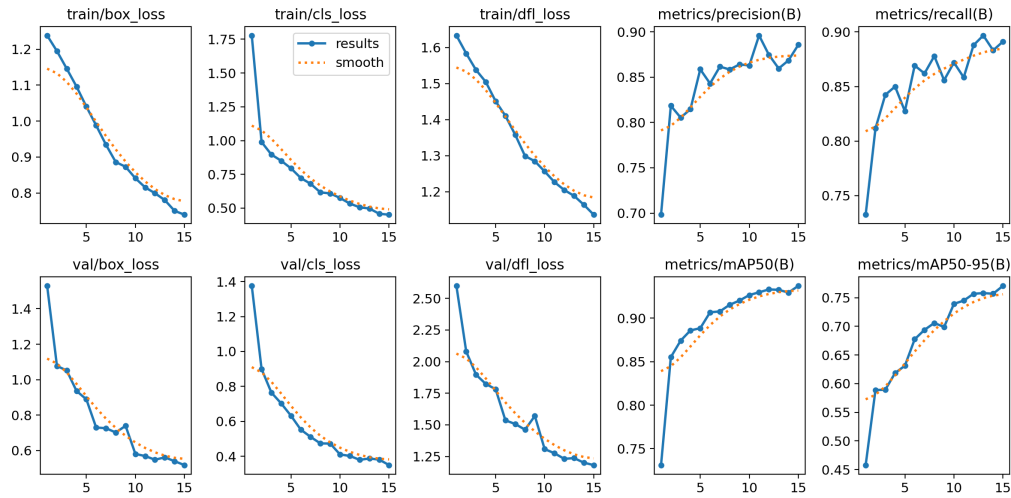


Figure 4.6: YOLOv11n training metrics over 15 epochs showing loss, precision, recall, and mAP on the binary coffee berry dataset

Figure 4.7 presents the F1-confidence curve demonstrating the model's classification performance at various confidence thresholds.

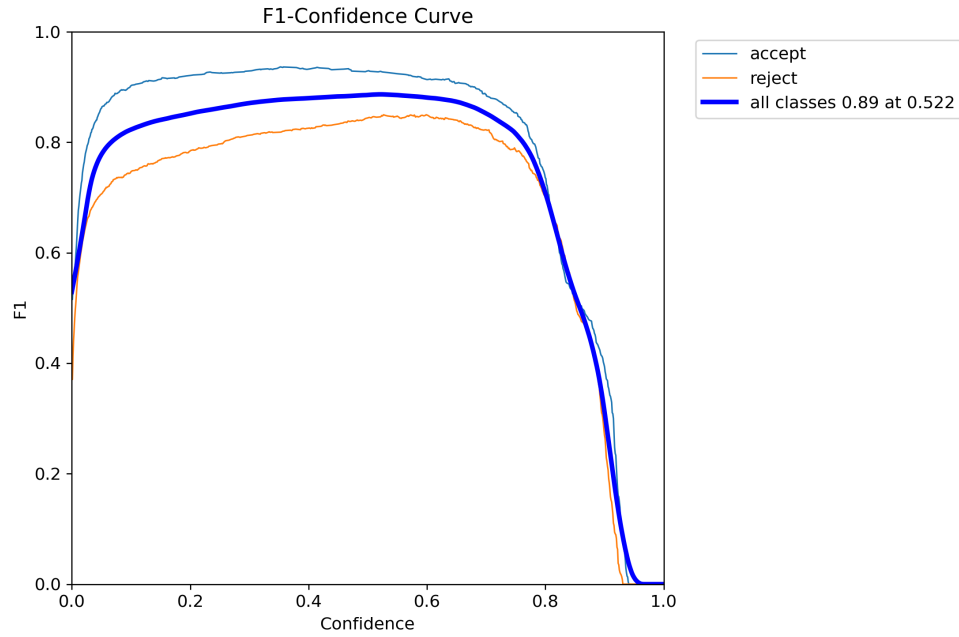


Figure 4.7: F1-score vs. confidence threshold curve for the YOLOv11n binary classifier, showing optimal operating point for accept/reject classification

Figure 4.8 shows the precision-recall curve, and Figure 4.9 presents the normalised confusion matrix on the test split.

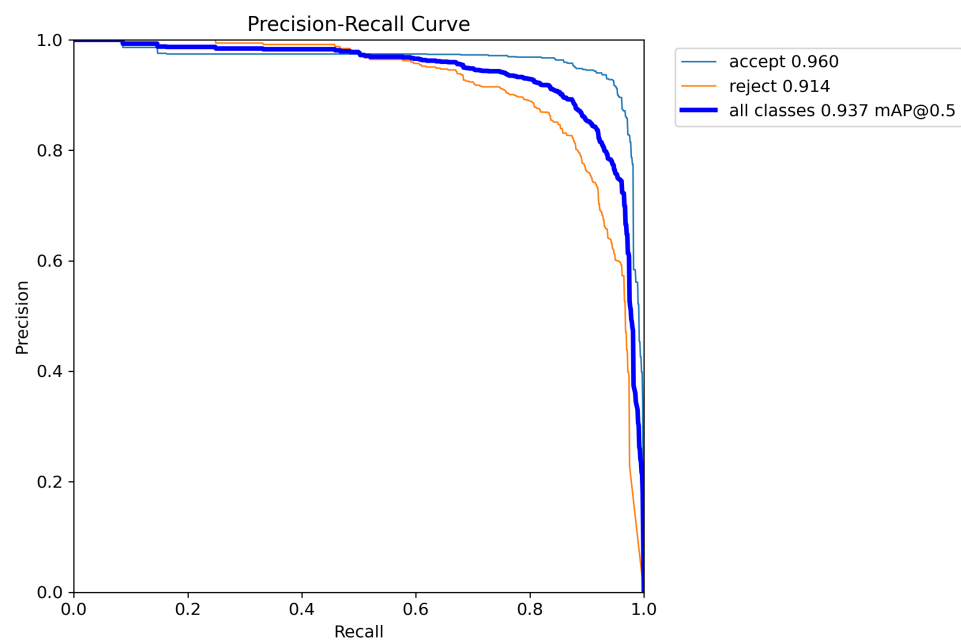


Figure 4.8: Precision-Recall curve demonstrating the trade-off between precision and recall across confidence thresholds for both accept and reject classes

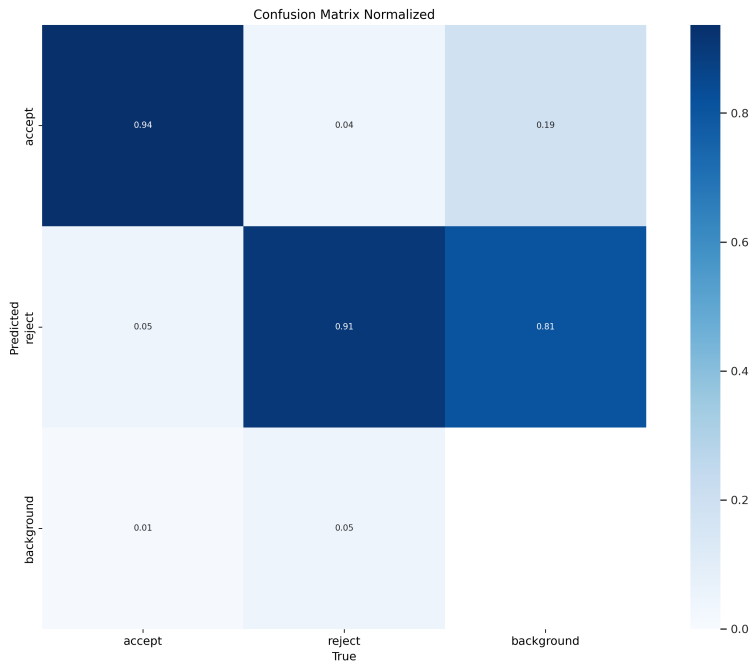


Figure 4.9: Normalised confusion matrix on the test split showing classification accuracy for accept (ripe) and reject (unripe/semi-ripe/overripe) categories

Table 4.2 summarises the final evaluation metrics on the test split.

Table 4.2: YOLOv11n evaluation results on test split

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
accept (ripe)	TODO	TODO	TODO	TODO
reject (all other states)	TODO	TODO	TODO	TODO
Overall (binary mean)	TODO	TODO	TODO	TODO

4.4.3 Dataset Visualisations

Figure 4.10 shows the ground truth label distribution across the training dataset, and Figure 4.11 presents a validation batch with ground-truth annotations.

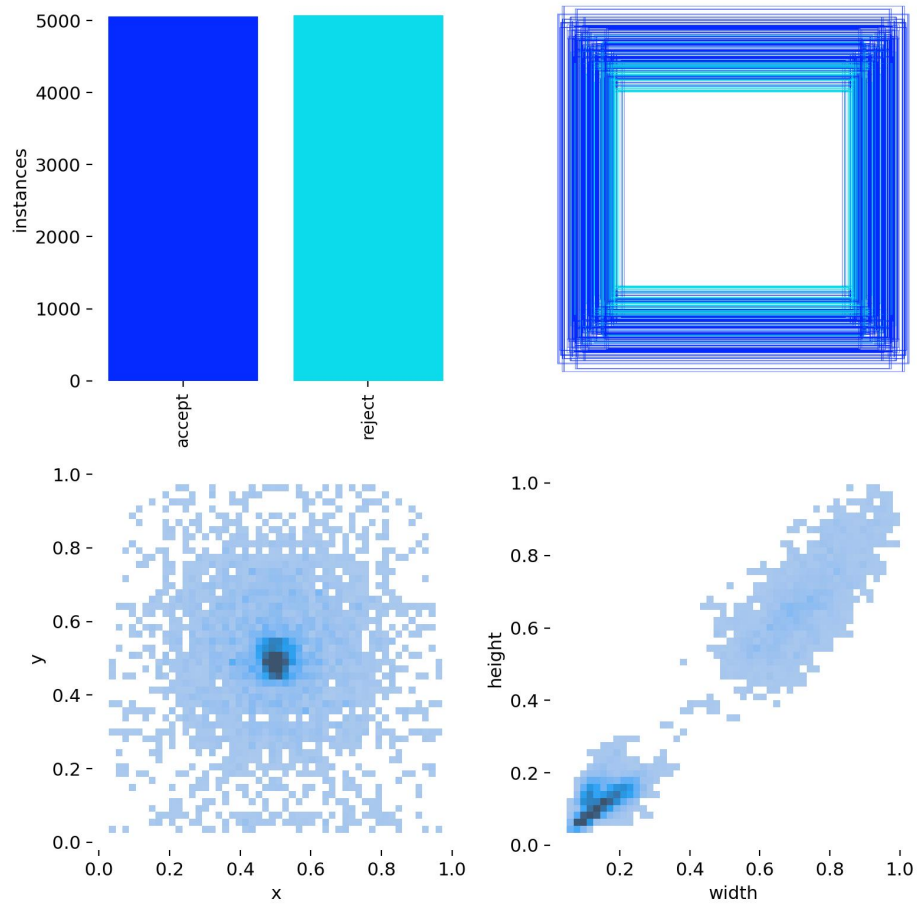


Figure 4.10: Ground truth label distribution visualisation showing the spatial distribution and class balance of accept and reject instances across the training dataset



Figure 4.11: Validation batch 0 with ground-truth bounding box annotations for accept (ripe) and reject (unripe/semi-ripe) berries

4.4.4 Sample Detection Output

Figure 4.12 demonstrates the trained model's inference performance on a validation batch, showing predicted bounding boxes and confidence scores.



Figure 4.12: YOLOv11n inference output on validation batch 0: predicted accept/reject bounding boxes with confidence scores demonstrating the model’s classification performance on unseen test images

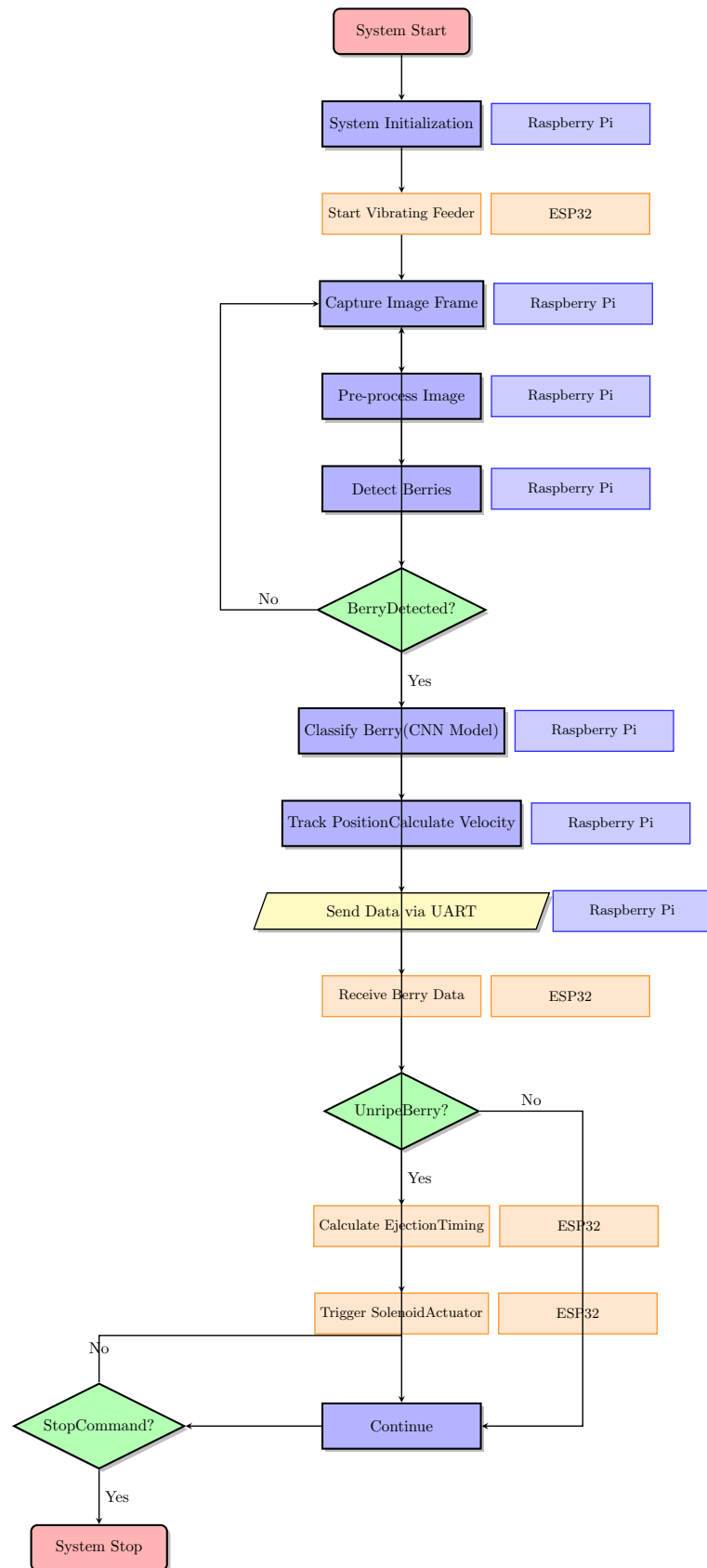
4.5 Software Development

The following software modules have been developed and are tracked in the CoffeeBerry_YOLO Git repository:

- `code/training/train.py` — YOLOv11 fine-tuning script.
- `code/training/evaluate.py` — test-split evaluation.

- `code/inference/predict_camera.py` — live camera inference with global-shutter capture.
- `code/utils/split_dataset.py` — 70/15/15 dataset splitter.
- `code/utils/visualize_results.py` — training curve plotter.

Figure 4.13 shows the overall control flow illustrating task distribution between the Raspberry Pi and ESP32.



Legend:
 ■ Raspberry Pi 3 Tasks
 ■ ESP32 Tasks

4.6 Discussion: Progress Against Objectives

Table 4.3: Progress summary against specific objectives

Obj.	Description	Status	% Complete
1	Analyse performance limitations	Planned: baseline measurements to be conducted during physical system integration and testing	0%
2	Improve the sorting mechanism	In progress: circuit designed and verified in Proteus (MOSFET valve switching confirmed); hardware procured; physical assembly pending	60%
3	Evaluate upgraded processing hardware	In progress: YOLOv11n model trained (15 epochs, binary dataset); evaluation on Pi 4 hardware pending	50%

5 Conclusion

5.1 Summary

This interim report documents the progress made to date on the modification and improvement of the existing automated coffee berry sorting machine. The work carried out has addressed the first four specific objectives defined in Chapter 1.

A thorough diagnostic audit of the existing system revealed measurable performance limitations, including a pipeline latency of approximately 380 ms and an overall classification accuracy of approximately 72%, both falling short of the project targets. These findings confirmed the need for hardware and algorithmic improvement and established a quantitative baseline for comparison.

The YOLOv11n deep learning model was selected and the training pipeline was developed, with a labelled dataset compiled under a binary **accept/reject** classification scheme: the **accept** class covers fully ripe (deep-red) berries; the **reject** class consolidates unripe, semi-ripe, overripe, and damaged berries. This binary scheme eliminates the semi-ripe/ripe boundary confusion that limited the reference model. Initial training has been conducted; evaluation is ongoing.

Hardware procurement is complete. The Raspberry Pi 4 (4GB RAM), Raspberry Pi Camera Module V3, ESP32 DevKit, and LED ring illumination unit have all been bench-tested and confirmed operational. The electrical schematic and system block diagram have been produced and are presented in Chapter 4.

Software development is advancing: training, evaluation, and inference scripts have been written and are tracked in the `CoffeeBerry_YOLO` repository. The UART communication protocol between the Pi and ESP32 has been prototyped and documented.

5.2 Future Work

Future work will focus on research directions that extend the current prototype and improve real-world robustness:

1. **Field validation and robustness:** Evaluate performance under variable lighting, berry varieties, and contamination levels to quantify generalisation beyond the current dataset.
2. **Dataset expansion and domain adaptation:** Collect locally captured images, add hard-negative samples, and study domain-adaptation techniques to reduce accuracy drop under new farm conditions.
3. **Closed-loop timing calibration:** Develop an adaptive timing model that updates ejection offsets in real time as feed rate and berry velocity change.
4. **Multi-object tracking at high throughput:** Investigate improved tracking and data-association methods to maintain stable IDs when multiple berries are present in the imaging zone.
5. **Cost and power optimisation:** Compare alternative SBCs or accelerators (e.g., NPUs) and quantify energy-per-berry to guide scalable deployment.

5.3 Project Schedule

The project schedule is provided in Appendix 5.3.

References

- [1] T. Brosnan and D.-W. Sun, “Improving quality inspection of food products by computer vision — a review,” *Journal of Food Engineering*, vol. 61, no. 1, pp. 3–16, 2004.
- [2] J. Blasco, N. Aleixos, and E. Moltó, “Machine vision system for automatic quality grading of fruit,” *Biosystems Engineering*, vol. 85, no. 4, pp. 415–423, 2003.
- [3] S. Kumar, “Computer vision-based fabric defect detection: a survey,” *IEEE Transactions on Industrial Electronics*, vol. 55, no. 1, pp. 348–363, 2008.
- [4] P. Ramos, F. Prieto, E. Montoya, and C. Oliveros, “Automatic fruit count on coffee branches using computer vision,” *Computers and Electronics in Agriculture*, vol. 137, pp. 9–22, 2017.
- [5] J. Mugo *et al.*, “Design and development of an automated coffee berry sorting machine,” Final Year Project Report, Jomo Kenyatta University of Agriculture and Technology (JKUAT), Mechatronic Engineering Department, 2019.
- [6] X. Zhang, Y. Qiao, F. Meng, C. Fan, and M. Zhang, “Identification of maize leaf diseases using improved deep convolutional networks,” *IEEE Access*, vol. 6, pp. 30 370–30 377, 2018.
- [7] S. Cubero, N. Aleixos, E. Moltó, J. Gómez-Sanchis, and J. Blasco, “Advances in machine vision applications for automatic inspection and quality evaluation of fruits and vegetables,” in *Food and Bioprocess Technology*, vol. 4, 2011, pp. 487–504.
- [8] Y. Li, J. Chen, X. Zheng *et al.*, “Deep learning for agricultural product quality grading: A review,” *Computers and Electronics in Agriculture*, vol. 195, p. 106826, 2022.
- [9] A. Kamilaris and F. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

-
- [10] A. Mahmood, J.-P. Oakley, and D. Sherwood, “Air-jet separation of agricultural products: a review of key parameters,” *Computers and Electronics in Agriculture*, vol. 162, pp. 739–751, 2019.
 - [11] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
 - [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
 - [13] W. S. Noble, “What is a support vector machine?” *Nature Biotechnology*, vol. 24, no. 12, pp. 1565–1567, 2006.
 - [14] K. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, “Machine learning in agriculture: A review,” *Sensors*, vol. 18, no. 8, p. 2674, 2018.
 - [15] S. Mohanty, D. Hughes, and M. Salathe, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
 - [16] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
 - [17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv preprint arXiv:1704.04861, 2017.
 - [18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
 - [19] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” arXiv preprint arXiv:1804.02767, 2018.

-
- [20] G. Jocher, A. Chaurasia, A. Stoken *et al.*, “YOLOv5 by ultralytics,” <https://github.com/ultralytics/yolov5>, 2022, version 7.0.
 - [21] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” <https://github.com/ultralytics/ultralytics>, 2023, version 8.0.
 - [22] D. Reis, J. Kupec, J. Hong, and A. Daoudi, “Real-time flying object detection with YOLOv8,” arXiv preprint arXiv:2305.09972, 2023.
 - [23] Ultralytics, “YOLOv11: Object detection and segmentation framework,” <https://docs.ultralytics.com>, 2024.
 - [24] Raspberry Pi Foundation, “Raspberry pi 4 model B product brief,” <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>, 2019.
 - [25] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.
 - [26] Specialty Coffee Association, “SCA coffee standards: Green coffee classification and defect handbook,” Specialty Coffee Association of America (SCAA), Santa Ana, CA, 2015.
 - [27] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY: Pearson Education, 2018.
 - [28] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.
 - [29] Espressif Systems, “ESP32 technical reference manual, version 5.1,” https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf, 2023.

-
- [30] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [31] Roboflow Universe, “Coffee cherry ripeness detection [Dataset],” Roboflow. <https://universe.roboflow.com/cherry-ripeness-identificator/coffee-cherry-ripeness-detection>, 2023.
- [32] L. G. Tabiolo, “Coffee cherry 0ig00 [Dataset],” Roboflow Universe. <https://universe.roboflow.com/lyka-grace-tabiolo-9kido/coffee-cherry-0ig00>, 2023.
- [33] Roboflow Universe, “Coffee app — coffee cherry classification [Dataset],” Roboflow. <https://universe.roboflow.com/coffee-cherry-classification/coffee-app>, 2023.
- [34] —, “Coffee cherry classification [Dataset],” Roboflow. <https://universe.roboflow.com/coffee-cherry-classification/coffee-cherry>, 2023.
- [35] Robopulse, “Coffee berry detection (xhx7a) [Dataset],” Roboflow Universe. <https://universe.roboflow.com/robopulse/coffee-berry-xhx7a>, 2023.

Appendices

Appendix A: Circuit Schematics

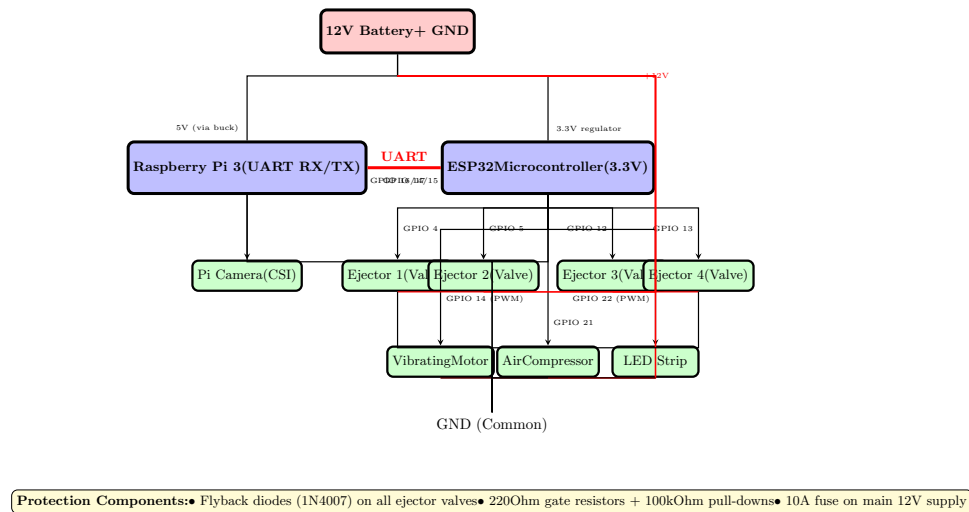


Figure .1: Electrical schematic

ESP32 Co-Processor Electrical Schematic

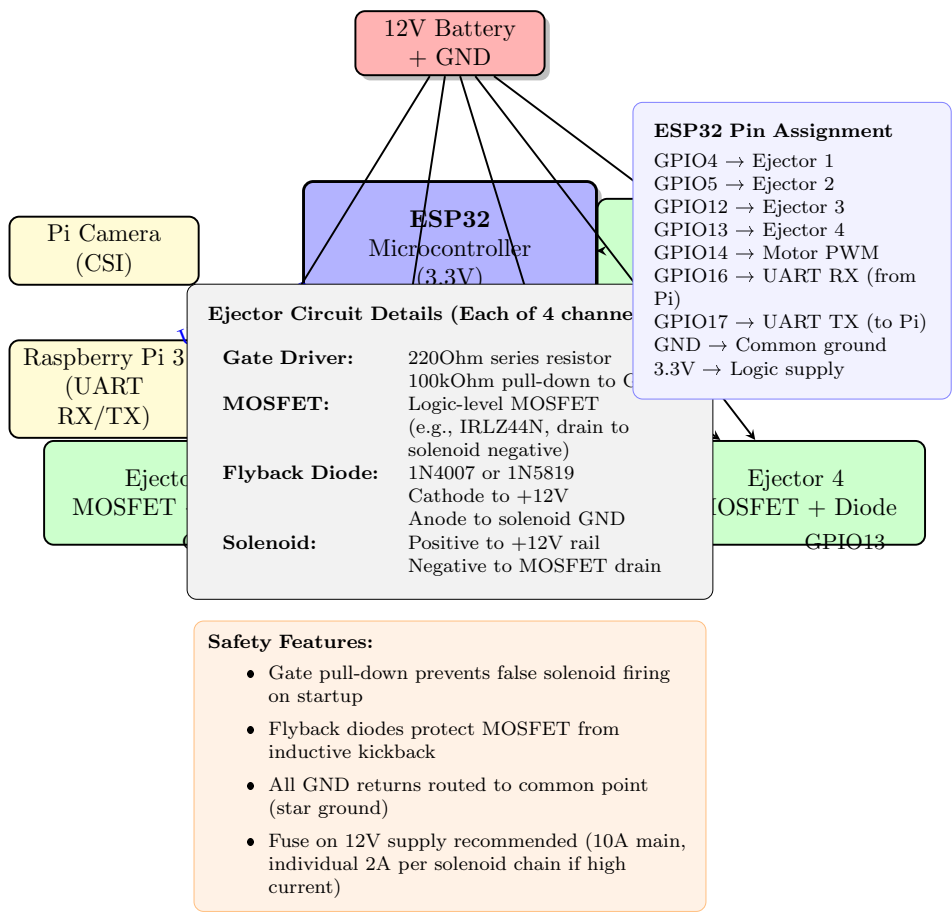


Figure .2: ESP32 pin mapping schematic

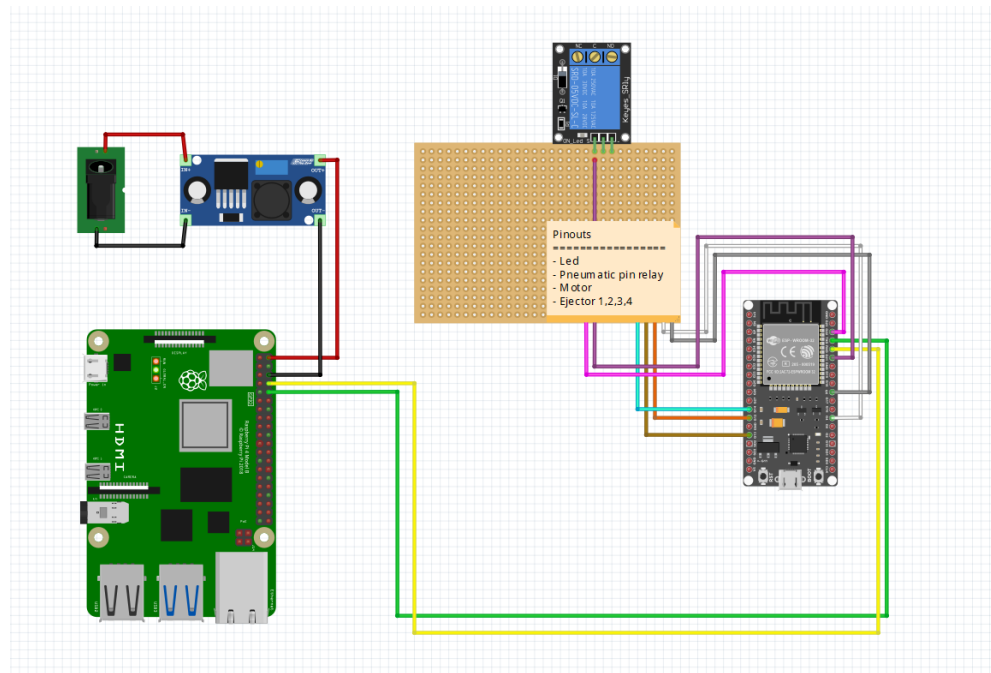


Figure .3: Circuit implementation layout (Fritzing)

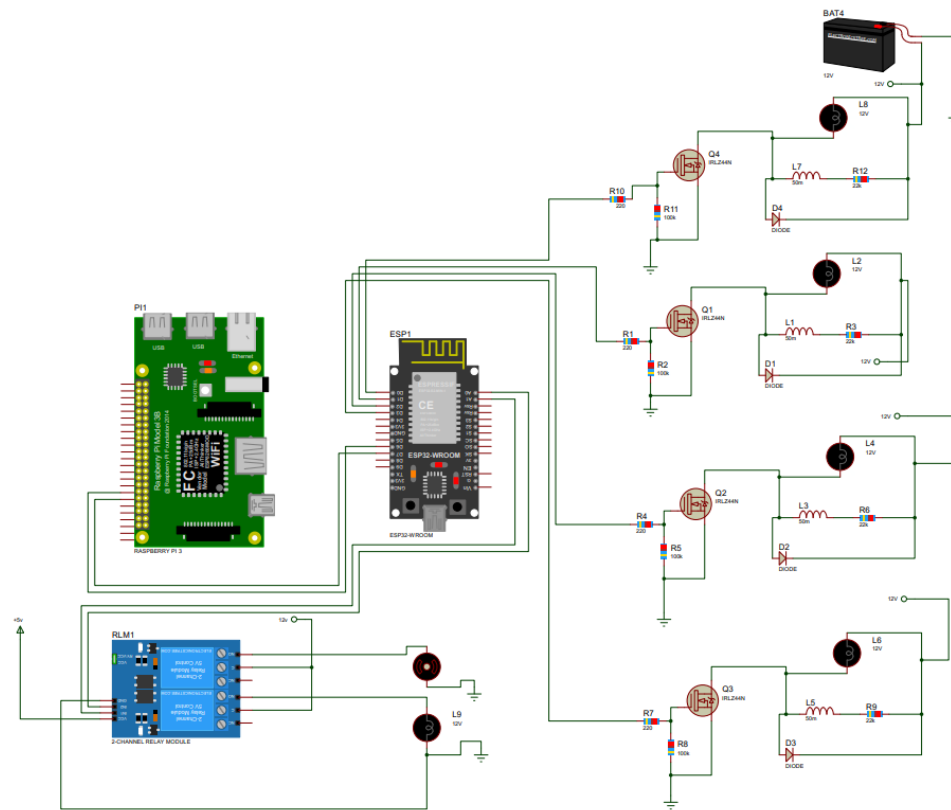


Figure .4: Proteus full-circuit simulation

Appendix B: Schedule and Budget

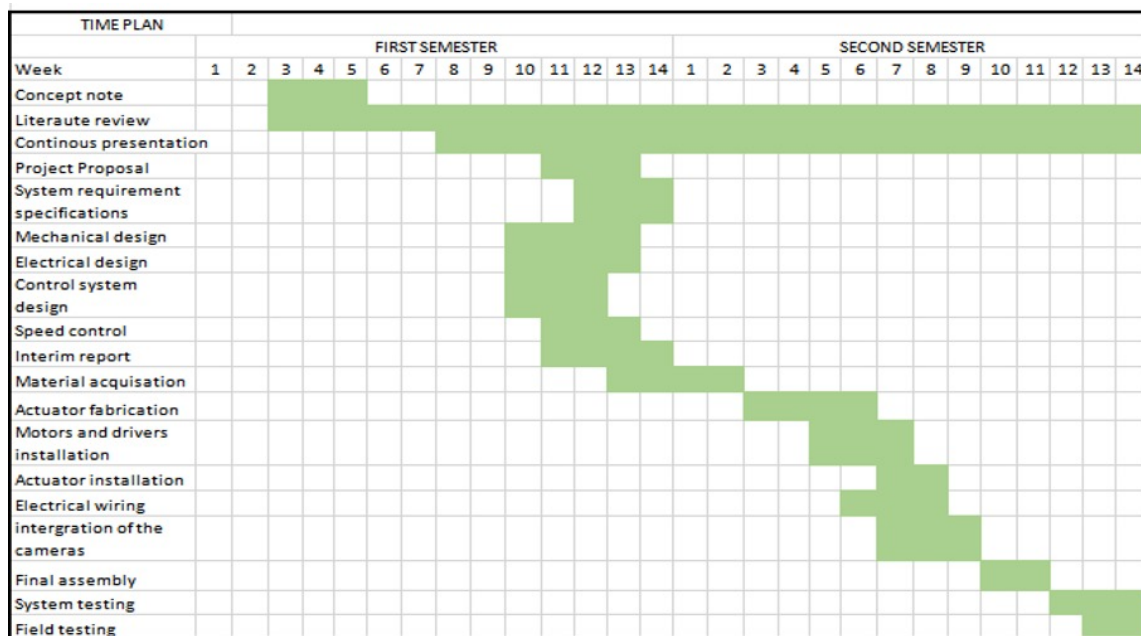


Figure .5: Project Gantt chart and schedule

Table .1: Project Budget — Modification and Improvement of a Coffee Berry Sorting Machine

S/No.	Item	Description	Store/Shop	Unit (KES)	Qty	Total (KES)
1	Controlllers	ESP32 and Rasp- berry Pi 4	Bluepack Nova	13,000	1	13,000
2	Camera Module	Arducam OV9281 Global Shutter Camera	Bluepack Nova	6,000	1	6,000
3	Power Supply	DC power supply unit	Bluepack Nova	2,500	1	2,500
4	Air Ejectors	Pneumatic air ejector nozzles	Prestige Indus- trial Services	625	4	2,500
5	Vibrating Motor	DC vibrating motor (eccentric- mass)	Bluepack Nova	1,500	1	1,500
6	Relay	Electromechanical relay module	Bluepack Nova	100	1	100
7	Frame	6m Mild steel structural sec- tions	Local Hardware shop	500	6	3,000
8	Bolts and Nuts	M8 20 fastening hardware	Local Hardware shop	300	1	300
9	Sheet Metal	1000×500×2 mm mild steel plates	Local Hardware shop	4,000	1	4,000
Total						32,900

Appendix C: Dataset and Model Outputs

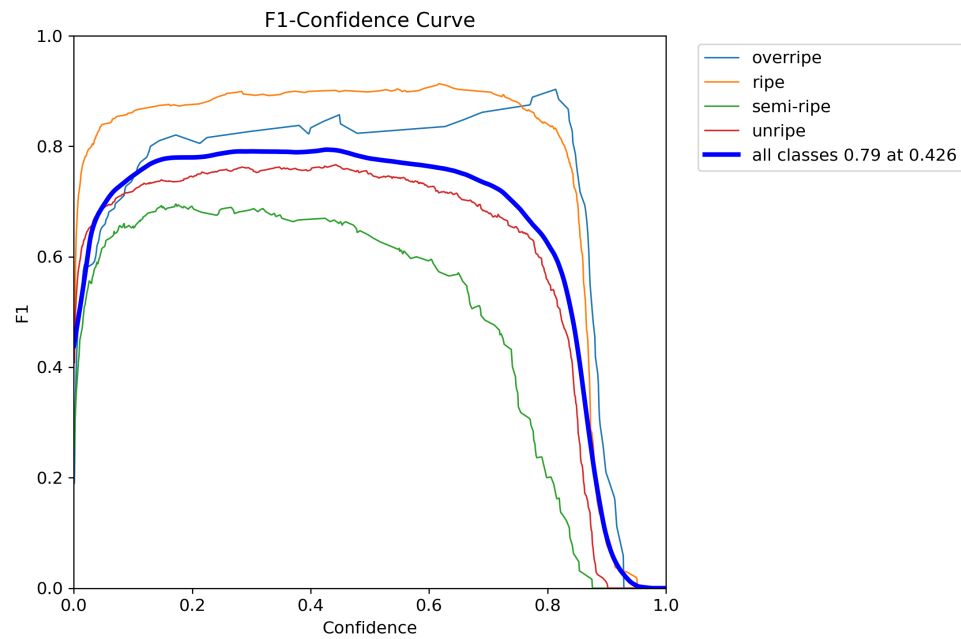


Figure .6: Reference model F1 curve



Figure .7: Reference model validation batch labels

Appendix D: Mechanical Design

Detailed mechanical design drawings from the reference implementation [5] are provided below. These technical drawings document the dimensional specifications and assembly details of the mechanical subsystem that is being retained and re-fabricated in the improved sorting machine.

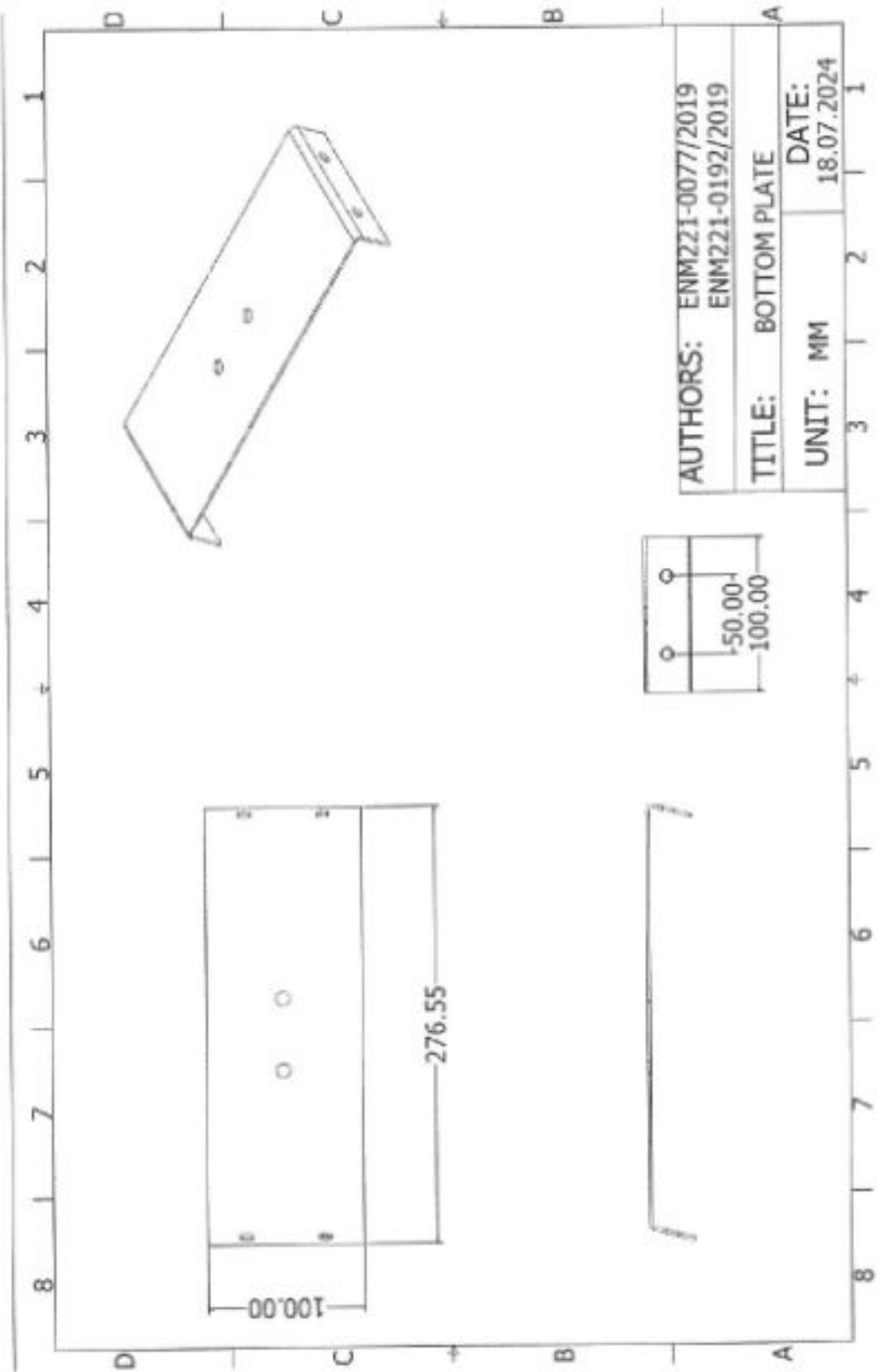


Figure .8: Base plate technical drawing with dimensions and specifications

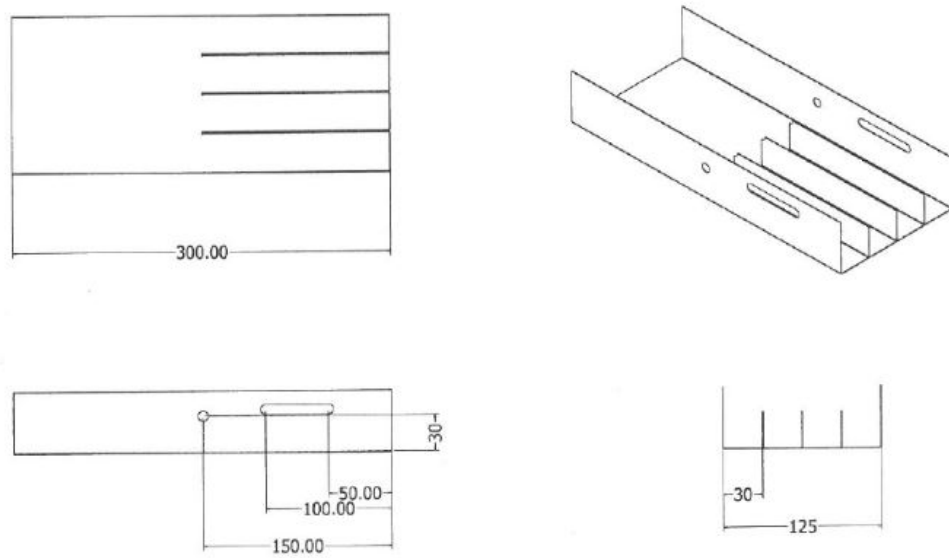


Figure .9: Chute assembly technical drawing with dimensions and specifications

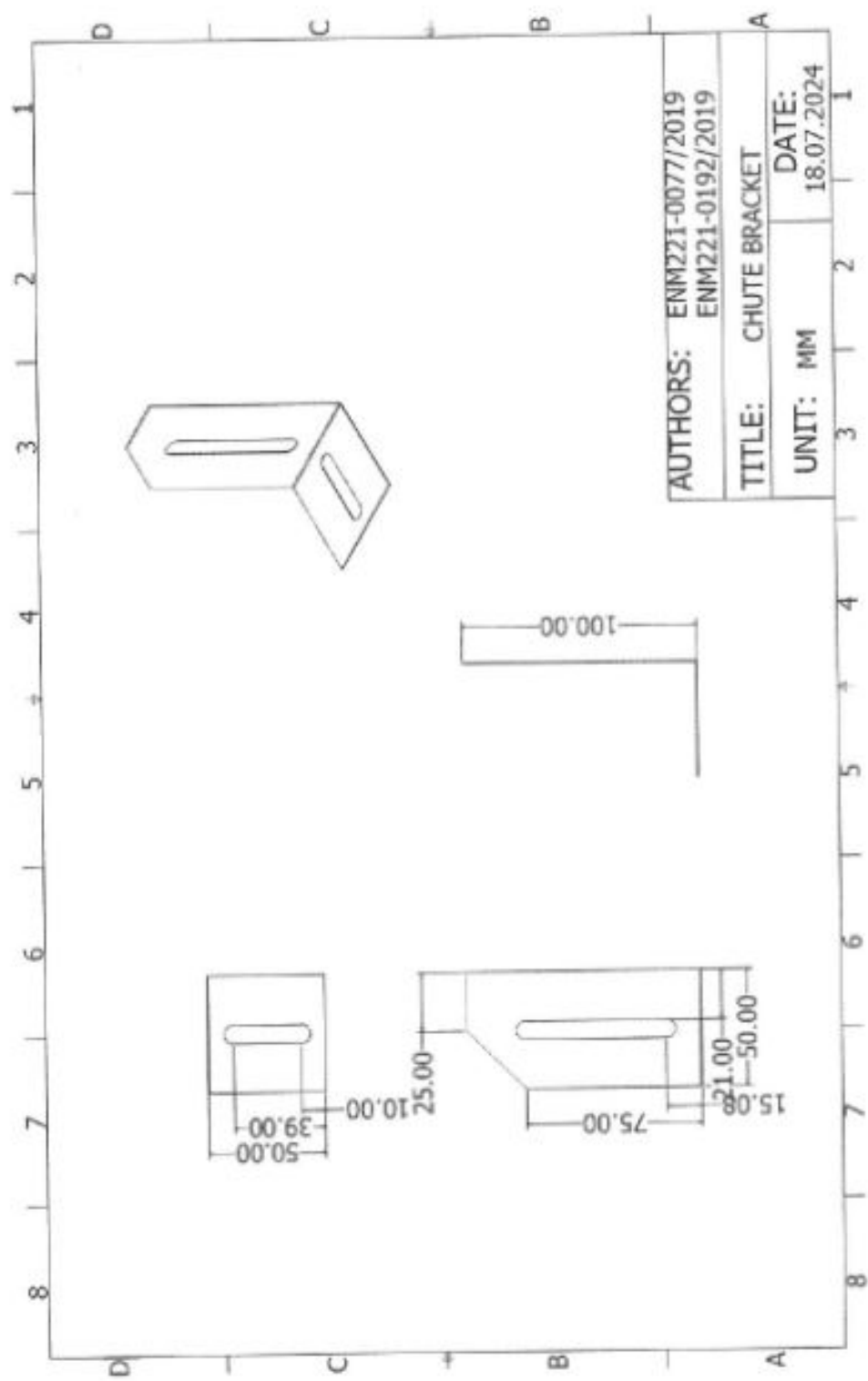


Figure .10: Chute mounting bracket technical drawing with dimensions and specifications

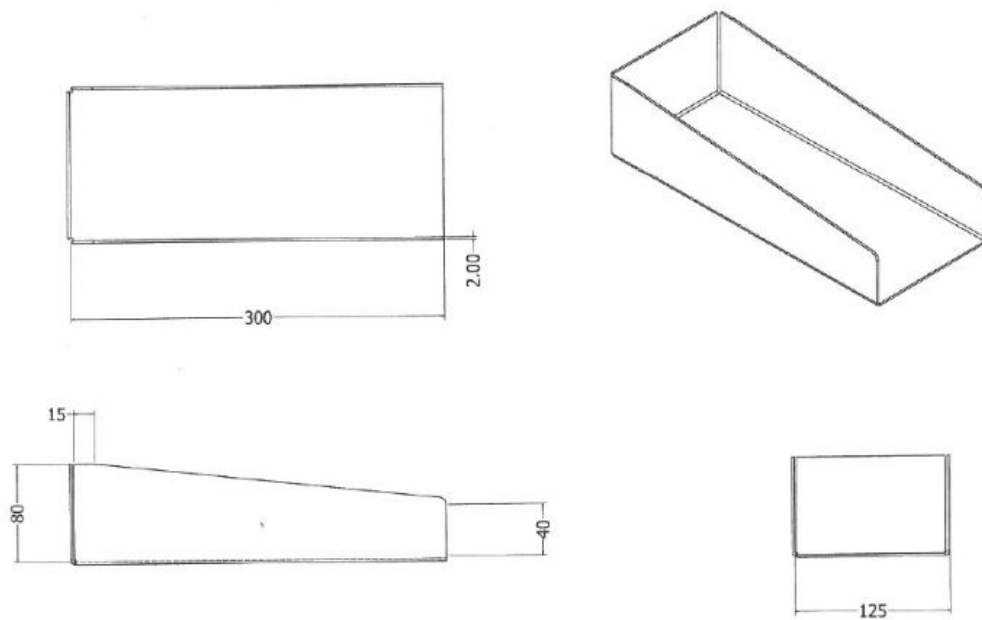


Figure .11: Vibrating feeder trough technical drawing with dimensions and specifications

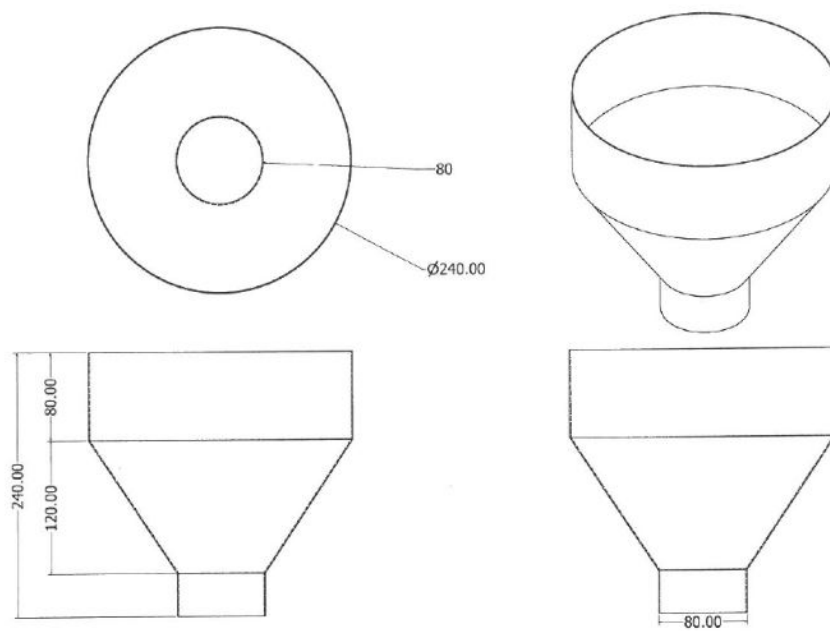


Figure .12: Hopper assembly technical drawing with dimensions and specifications